

ARAIM 测试

2026-06-02

1. 简介

ARAIM 测试

- ARAIM 测试
- test_araim, test_predicted_araim, test_integrity_fusion_policy, test_integrity_snapshot, test_lidar_observability_fim, test_pl_grid, test_future_pl_field_predictor

ARAIM 测试 snapshot, LiDAR FIM, PL grid 测试 PL 测试 demo 测试 launch 测试 ARAIM 测试 ARAIM 测试

2. 测试

测试	测试	测试
test_araim	79	GNSS ARAIM, LiDAR ARAIM, IntegrityMonitor, TrunkMap, fault hypothesis, golden-output regression, degenerate geometry regression, fault injection regression, H/V safety semantics validation
test_predicted_araim	5	open-sky, fallback, legacy wrapper, epoch, clock Schur complement
test_integrity_fusion_policy	13	GNSS/LiDAR/fallback fusion policy
test_integrity_snapshot	4	snapshot, GNSS epoch, LiDAR, current fields copy
test_lidar_observability_fim	13	LiDAR observability/FIM, fallback, PCA/voxel/normal
test_pl_grid	5	PL grid indexing, corner, fallback reason, FIM diagnostics
test_future_pl_field_predictor	11	grid/direct, GNSS-only, fused mode, advisory FIM, LiDAR FIM

3. 测试

iap 测试

```
source install/setup.bash
colcon build --packages-select iap --cmake-args -DBUILD_TESTING=ON
```

ARAIM 测试

```
source install/setup.bash
ctest --test-dir build/iap -R
'test_araim|test_predicted_araim|test_integrity_snapshot|test_future_pl_field_predictor|test_lidar_observability_fim|test_pl_grid|test_integrity_fusion_policy' --output-on-failure
```

gtest 测试

```
./build/iap/test_araim --gtest_list_tests
./build/iap/test_predicted_araim --gtest_list_tests
./build/iap/test_integrity_fusion_policy --gtest_list_tests
./build/iap/test_integrity_snapshot --gtest_list_tests
./build/iap/test_lidar_observability_fim --gtest_list_tests
./build/iap/test_pl_grid --gtest_list_tests
./build/iap/test_future_pl_field_predictor --gtest_list_tests
```

4. 测试

测试

```
Starting >>> iap
Finished <<< iap [1.60s]

Summary: 1 package finished [1.66s]
```

测试

Test Name	Result	Time
test_araim	Passed	0.09 sec
test_integrity_fusion_policy	Passed	0.05 sec
test_predicted_araim	Passed	0.05 sec
test_integrity_snapshot	Passed	0.05 sec
test_lidar_observability_fim	Passed	0.05 sec
test_pl_grid	Passed	0.05 sec
test_future_pl_field_predictor	Passed	0.05 sec

```
100% tests passed, 0 tests failed out of 7

Label Time Summary:
gtest      =   0.38 sec*proc (7 tests)

Total Test time (real) =   0.39 sec
```

5. Build

- test_araim and test_future_pl_field_predictor use /config.json for timing/logging configuration
- Build ARAIM using CMake and run 3 tests

6. Golden-output regression test GNSS

Use C++

```
AraimGoldenTest.FaultFreeSyntheticSixSatelliteGeometry
```

```
src/iap/test/test_araim.cpp
```

```
src/iap/test/araim_validation/reference_gnss_wls_pl.py
src/iap/test/araim_validation/araim_reference_output.txt
```

6.1 Setup

Use C++ GnssAraimEvaluator::runLinearized() for synthetic GNSS epoch and Python WLS reference for fault-free protection level and golden-output regression test. ARAIM uses K factor and HPL for fault-free PL.

6.2 Data

Use 6 GPS satellites

PRN	Constellation	Azimuth	Elevation	Sigma	Residual
1	GPS	0 deg	60 deg	1.5 m	0 m
2	GPS	60 deg	50 deg	1.5 m	0 m
3	GPS	120 deg	55 deg	1.5 m	0 m
4	GPS	200 deg	45 deg	1.5 m	0 m
5	GPS	280 deg	50 deg	1.5 m	0 m
6	GPS	330 deg	65 deg	1.5 m	0 m

C++ uses GnssAraimLinearizedInput

- Python reference uses 6x4 GNSS linearized geometry matrix
- W = 1 / 1.5^2 = 0.444444444444...
- r = 6

- `prns=[1, 2, 3, 4, 5, 6]`
- `constellation_ids=[0, 0, 0, 0, 0, 0]` GPS

6.3

ARAIM

P_HMI_req	1e-7	PHMI requirement
P_FA_req	1e-5	false alarm requirement
p_sat_default	1e-5	fault prior
p_const_GPS	1e-4	GPS constellation fault prior
dynamic_budget	false	Python reference <code>Kff</code>
K_ff	5.451310438136472	Python reference <code>Q(PHMI_alloc_0 / 2)</code>
K_fa	0.0	neutralize always-on satellite subset contribution
K_md	0.0	neutralize always-on satellite subset contribution
enable_constellation_faults	false	golden test fault-free PL
enable_trunk_hypotheses	false	trunk placeholder
parallel_hypotheses	false	

C++ ARAIM production code single-satellite GNSS fault hypotheses production code golden test fault-free PL `K_fa` `K_md` `0.0` residual subset term PL

6.4 Golden reference

Python reference C++ `EXPECT_NEAR(..., 1e-6)`

	Reference
Kff	5.451310438136472
sigma_e	1.460597699646529
sigma_n	2.044854590757374
sigma_u	12.528175678508841
PL_E	7.962171486001244
PL_N	11.147137175066957
PL_U	68.29497484706273
HPL	11.147137175066957
VPL	68.29497484706273

C++

- `result.valid == true`
- `result.n_hypotheses == 6`
- `result.n_detected == 0`
- `result.has_degenerate_hypothesis == false`
- `result.HPL == max(result.PL_E, result.PL_N)`
- `result.VPL == result.PL_U`

6.5

`test_araim`

```
source install/setup.bash
colcon test --packages-select iap --ctest-args -R "test_araim" --output-on-failure
```

```
Starting >>> iap
Finished <<< iap [0.08s]

Summary: 1 package finished [0.16s]
```

golden test

```
source install/setup.bash
./build/iap/test_araim --gtest_filter=AraimGoldenTest.FaultFreeSyntheticSixSatelliteGeometry --gtest_brief=1
```

```
[=====] 1 test from 1 test suite ran. (0 ms total)
[ PASSED ] 1 test.
```

6.6 Fixed GNSS geometry

fault-free geometry Python reference C++ golden test G/W/r

	Python reference	C++ golden test expected	
Kff	5.451310438136472	5.451310438136472	
HPL	11.147137175066957	11.147137175066957	
VPL	68.29497484706273	68.29497484706273	

C++ EXPECT_NEAR(..., 1e-6) sigma_ff_E/N/U PL_E/N/U HPL VPL fixed GNSS geometry fault-free C++ GnsAraimEvaluator::runLinearized() HPL/VPL Python WLS reference

golden test fault-free PL K_fa=0.0 K_md=0.0 C++ always-on single-satellite subset terms PL fault injection test K_fa=4.5 fault injection bias=0 HPL/VPL golden reference faulted subset threshold term PL golden

golden-output regression test GNSS fault-free golden test C++ ARAIM fault-free WLS PL Python reference

7. Degenerate geometry regression

C++

```
AraimDegenerateTest.TooFewSatellitesDoesNotProduceNan
AraimDegenerateTest.NearlySingularGeometryDoesNotProduceNan
AraimDegenerateTest.InvalidWeightsAreRejected
```

```
src/iap/test/test_araim.cpp
```

7.1

GnsAraimEvaluator::runLinearized() GNSS NaN/Inf PL GnsAraimResult failure_reason

- result.valid == false
- HPL VPL pl_araim vpl_araim finite
- HPL VPL NaN/Inf
- HPL VPL conservative sentinel

7.2

TooFewSatellitesDoesNotProduceNan	3 G/W/r min_sats=4	invalid HPL/VPL finite conservative NaN/Inf
NearlySingularGeometryDoesNotProduceNan	5 (az,el)=(0,30),(5,31),(10,32),(15,33),(20,34)	eps_degen=1e-3 nominal factorization invalid NaN/Inf
InvalidWeightsAreRejected	5 3 weight NaN/Inf 0-1	subcase invalid HPL/VPL finite conservative NaN/Inf

sigma runLinearized() W=1/sigma^2 sigmas_m W sigma/variance linearized solver

7.3

degenerate tests

10

```
test_araim
```

111

gtest XML

degenerate geometry regression tests `test_araim` 74 `gtest` 3 measurement weights

8. Fault injection regression PRN 3 residual bias

□□ C++ □□□

5/5

8.1 背景

ARAIM residual fault golden test GNSS geometry PRN 3 residual bias

□□□□□□ bias □ HPL/VPL □□□□□□ golden □□□□□□□□□□

- `bias` `valid` `finite` `NaN`/`Inf`
- PRN 3 `subset` `d_horiz` `d_vert` `bias`
- `bias=0` PRN
- `bias=20` `bias=0` fault detection
- `excluded_prns` PRN 3

8.2 □□□□

golden test

PRN	Azimuth	Elevation	Sigma
1	0 deg	60 deg	1.5 m
2	60 deg	50 deg	1.5 m
3	120 deg	55 deg	1.5 m

PRN	Azimuth	Elevation	Sigma
4	200 deg	45 deg	1.5 m
5	280 deg	50 deg	1.5 m
6	330 deg	65 deg	1.5 m

ARAIM 配置

dynamic_budget	false
K_ff	5.451310438136472
K_fa	4.5
K_md	0.0
enable_constellation_faults	false
enable_trunk_hypotheses	false
parallel_hypotheses	false

K_md=0.0 故障検出/分離 K_fa=4.5 検出閾値

8.3 実行

AraimFaultInjectionTest の gtest XML 出力

Bias	HPL		VPL	PRN3 d_horiz	PRN3 d_vert	n_detected	n_hypotheses	worst_h
0 m	15.450104898584801	97.704623707250704		0	0	0	6	
1 m	16.108003696797525	101.8650973699709	0.88450432427255521	3.4300150266581531		0	6	
3 m	17.423801293222962	110.18604469541121	2.6535129728176652	10.290045079974453		0	6	
5 m	18.73959888964842	118.5069920208517	4.4225216213627707	17.150075133290706		0	6	
10 m	22.029092880712042	139.30936033445266	8.8450432427255414	34.300150266581412		0	6	
20 m	28.60808086283928	180.91409696165465	17.690086485451083	68.600300533162823		3	6	

注

- PRN3 d_horiz と d_vert は 0 m から 20 m までの範囲
- HPL/VPL の bias は 0 から bias=0 の範囲
- bias=20 m の場合、故障検出時に excluded_prns に PRN 3 が含まれる
- 無限大は NaN/Inf

8.4 実行結果

故障注入テストの結果

```
source install/setup.bash
./build/iap/test_araim --gtest_filter='AraimFaultInjectionTest.*' --gtest_brief=1
```

出力

```
[=====] 1 test from 1 test suite ran. (0 ms total)
[ PASSED ] 1 test.
```

実行コマンド test_araim

```
source install/setup.bash
colcon test --packages-select iap --ctest-args -R "test_araim" --output-on-failure
```

注

```
Starting >>> iap
Finished <<< iap [0.08s]

Summary: 1 package finished [0.17s]
```

gtest XML

```
test_araim: 75 tests, 0 failures, 0 errors
AraimFaultInjectionTest: 1 test, 0 failures, 0 errors
```

fault injection regression test `test_araim` 75 gtest PRN 3 residual bias ARAIM fault separation/detection/exclusion

9. Fault injection CSV/plot

```
src/iap/scripts/araim_validation/export_fault_injection_results.py
```

Reference canonical

```
src/iap/scripts/araim_validation/reference_gnss_wls_pl.py
```

```
src/iap/test/araim_validation/reference_gnss_wls_pl.py
```

9.1

pass/fail fault injection `test_araim.gtest.xml` `AraimFaultInjectionTest` gtest properties CSV

- HPL/VPL vs injected bias
- detected fault count vs injected bias

`pandas` Python `csv` `matplotlib` `pandas`

9.2

`test_araim` gtest XML

```
source install/setup.bash
colcon test --packages-select iap --ctest-args -R "test_araim" --output-on-failure
```

CSV

```
python3 src/iap/scripts/araim_validation/export_fault_injection_results.py
```

```
build/iap/test_results/iap/test_araim.gtest.xml
```

```
src/iap/results/araim_validation
```

9.3

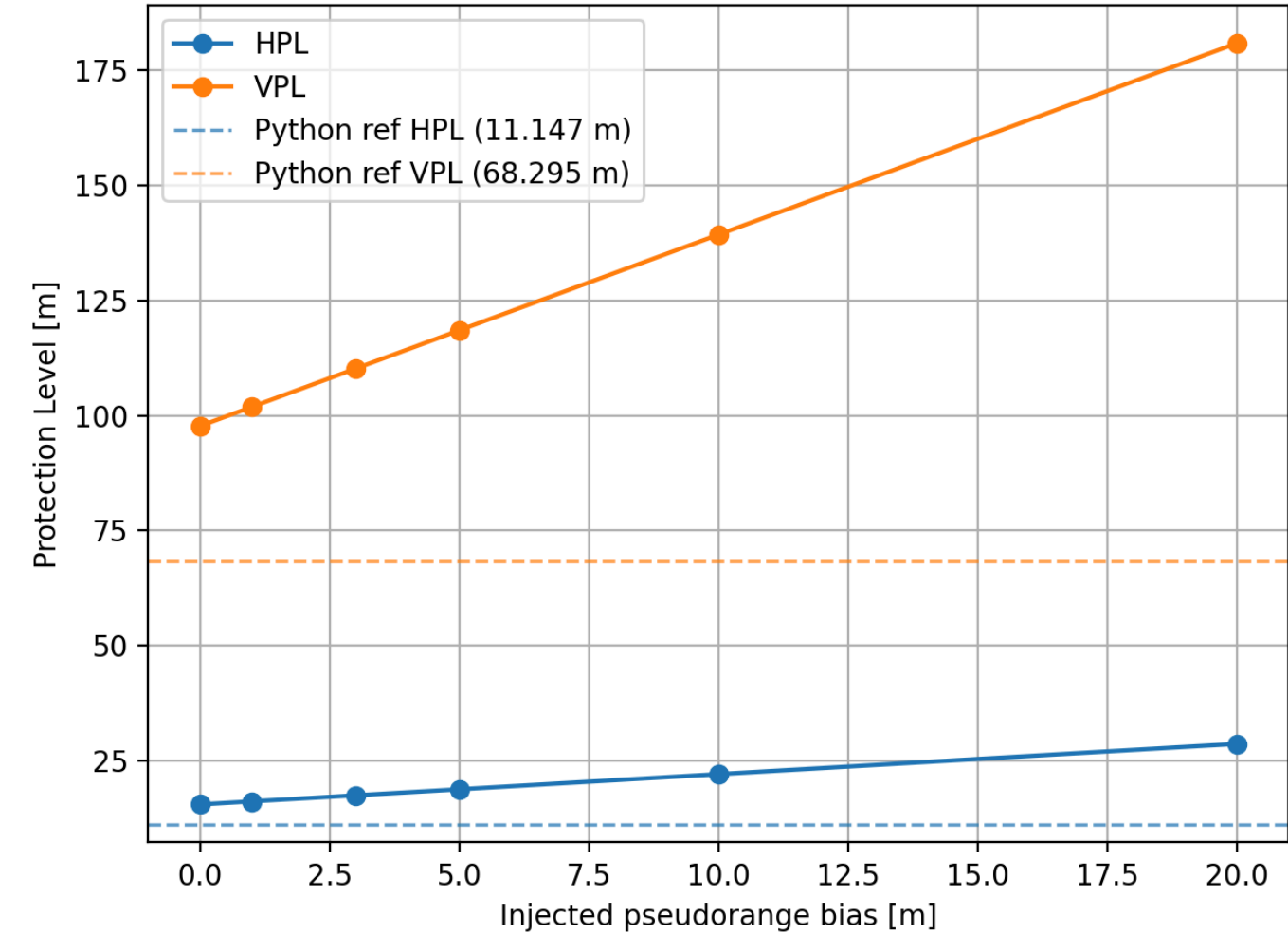
```
src/iap/results/araim_validation/gnss_fault_injection.csv
src/iap/results/araim_validation/pl_vs_bias.png
src/iap/results/araim_validation/detected_fault_vs_bias.png
```

00000000

00	00
gnss_fault_injection.csv	0000643 bytes
pl_vs_bias.png	0000PNG 1280x960074571 bytes
detected_fault_vs_bias.png	0000PNG 1280x960056482 bytes

9.4 0000

HPL/VPL vs injected bias



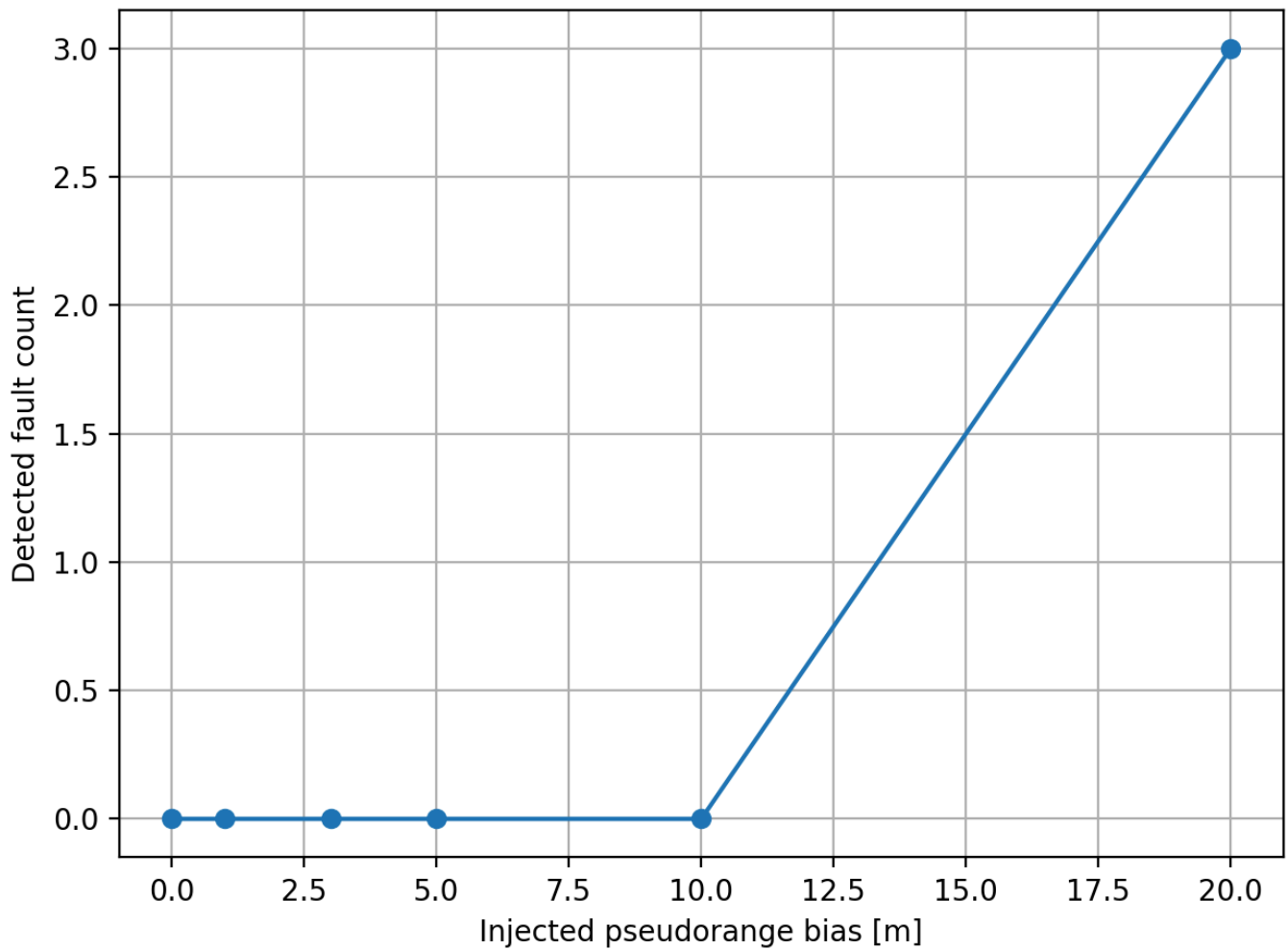
00000000 fault injection test 0 K_fa=4.50K_md=0.0 0000 ARAIM HPL/VPL0000 Python fault-free reference0

- Python reference HPL = 11.147137175067 m
- Python reference VPL = 68.294974847063 m

0000 6 0 golden-output regression 0 C++ expected HPL/VPL 00000000 fixed GNSS geometry 0 fault-free HPL/VPL 0 Python reference 00000000 bias=0
0000000000 fault injection test 000 K_fa=4.5 0 single-satellite subset threshold term00 golden/reference 00000 K_fa=0.00K_md=0.0 000 fault-free
PL-000000000

0000 injected pseudorange bias 0 0 m 000 20 m 00HPL 15.4501 m 000 28.6081 m0VPL 97.7046 m 000 180.9141 m0VPL 0000 HPL0000000000
vertical dilution 000000000000000000 PRN 3 residual bias 00 ARAIM 000 protection level020 m 0 bias 0 0 m baseline 00000000

Detected fault count vs injected bias



0/1/3/5/10 m bias gnss_n_det=0 20 m bias gnss_n_det=3 K_fa=4.5 bias solution separation PL
detection threshold 20 m CSV excluded_prn=3 fault PRN 3

fault injection "PL bias " bias detection/exclusion" NaN/Inf invalid

9.5 CSV

```
bias_m,hpl,vpl,gnss_valid,gnss_n_hyp,gnss_n_det,excluded_prn,worst_hypothesis,failure_reason,prn3_d_horiz,prn3_d_vert
0,15.450104898584801,97.704623707250704,1,6,0,,1,none,0,0
1,16.108003696797525,101.8650973699709,1,6,0,,1,none,0.88450432427255521,3.4300150266581531
3,17.423801293222962,110.18604469541121,1,6,0,,1,none,2.6535129728176652,10.290045079974453
5,18.73959888964842,118.5069920208517,1,6,0,,1,none,4.4225216213627707,17.150075133290706
10,22.029092880712042,139.30936033445266,1,6,0,,1,none,8.8450432427255414,34.300150266581412
20,28.60808086283928,180.91409696165465,1,6,3,3,1,none,17.690086485451083,68.600300533162823
```

9.6

ARAIM validation

```
src/iap/scripts/araim_validation/reference_gnss_wls_pl.py
src/iap/scripts/araim_validation/export_fault_injection_results.py
src/iap/test/test_araim.cpp
src/iap/results/araim_validation/gnss_fault_injection.csv
src/iap/results/araim_validation/pl_vs_bias.png
src/iap/results/araim_validation/detected_fault_vs_bias.png
```

```
source install/setup.bash
colcon build --symlink-install --packages-select iap \
  --cmake-args -DCMAKE_BUILD_TYPE=RelWithDebInfo -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
colcon test --packages-select iap --ctest-args -R "test_araim" --output-on-failure
```

000

- ## 10. H/V safety semantics validation

10.1 □□□□

- integrity monitor
- horizontal/vertical safety semantics
- GNSS ARAIM
- planner
- ROS launch

- ## 10.2 練習問題

10.3 練習問題

11

- #### 10.4 H/V four-case safety table

Case	HPL	HAL	VPL	VAL	im_h	im_v	IM	Expected state	Result
both safe	5	10	6	10	5	4	4	SAFE	PASS
horizontal unsafe	12	10	6	10	-2	4	-2	UNSAFE	PASS
vertical unsafe	5	10	12	10	5	-2	-2	UNSAFE	PASS
both unsafe	12	10	12	10	-2	-2	-2	UNSAFE	PASS

monitor recovery_count=1nominal_fraction=1.0 H/V safety semantics recovery hysteresis both-safe case UNSAFE

10.5 Vertical violation regression

```
HPL=5, HAL=10, VPL=10, VAL=10
```

- HPL < HALhorizontal safePASS
- VPL >= VALvertical unsafePASS
- state == UNSAFEPASS
- im_v <= 0PASS
- IM == im_vPASS

vertical violation SAFEUNSAFE

10.6 Equality boundary tests

Boundary	Input	Expected	Result
horizontal equality	HPL=HAL=10, VPL=6, VAL=10	im_h=0UNSAFE	PASS
vertical equality	HPL=5, HAL=10, VPL=VAL=10	im_v=0UNSAFE	PASS

alert limit equality unsafe strict < safety semantics

10.7 ROS message mapping

IntegrityReportMappingTest.HvMarginsMapToRosMessage internal IntegrityReport

```
fill_integrity_report_msg(report, msg)
```

Mapping	Result
msg.im_h == report.im_h	PASS
msg.im_v == report.im_v	PASS
msg.im_min == report.IM	PASS
msg.im == msg.im_min	PASS

legacy msg.im im_minROS message mapping H/V margin semantics

10.8 Build/test results

```
source install/setup.bash
colcon build --symlink-install --packages-select iap \
  --cmake-args -DCMAKE_BUILD_TYPE=RelWithDebInfo -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
```

```
Summary: 1 package finished [10.6s]
```

stderr deprecated API warnings H/V tests

~~~~~

```
source install/setup.bash
colcon test --packages-select iap \
  --ctest-args -R "test_integrity_monitor|test_araim" --output-on-failure
```

~~~

```
Starting >>> iap
Finished <<< iap [0.08s]

Summary: 1 package finished [0.17s]
```

~~~~~

```
./build/iap/test_araim --gtest_filter='IntegrityMonitorHvSafetyTest.*:IntegrityReportMappingTest.*' --gtest_brief=1
```

~~~

```
[=====] 4 tests from 2 test suites ran. (0 ms total)
[ PASSED ] 4 tests.
```

test_araim.gtest.xml

```
test_araim: 79 tests, 0 failures, 0 errors
IntegrityMonitorHvSafetyTest: 3 tests, 0 failures, 0 errors
IntegrityReportMappingTest: 1 test, 0 failures, 0 errors
```

colcon test-result --all

```
Summary: 592 tests, 3 errors, 396 failures, 15 skipped
```

~~~~~ bspline\_opt ego\_planner path\_searching ~~~~ iap ~~~~~~ iap ~~~~~~ 0 failures / 0 errors

10.9 Final conclusion

- H/V margin im\_h=HAL-HPL im\_v=VAL-VPL IM=min(im\_h,im\_v)
- SAFE H/V alert limit
- horizontal violation vertical violation both violation UNSAFE
- equality at alert limit UNSAFE
- vertical violation SAFE
- ROS msg.im msg.im\_min msg.im\_h/msg.im\_v/msg.im\_min internal report

11. GNSS / LiDAR / fallback / fusion policy validation

IntegrityFusionPolicy IntegrityReport -> msg/IntegrityReport GNSS ARAIM LiDAR geometry/FIM planner

11.1 Files modified

| File                                          | Change                                                                                                               |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| src/iap/test/test_integrity_fusion_policy.cpp | fusion policy unit tests source-only max_pl disabled/invalid required source conservative finite weighted_debug_only |
| src/iap/test/test_araim.cpp                   | IntegrityReportMappingTest source/fusion/failure fields ROS message                                                  |
| src/iap/docs/dev_ARAIM/ARAIM_test.md          |                                                                                                                      |

~~~~~ GNSS ARAIM formulas LiDAR formulas planner code ROS topic names CMake target structure Araim /

11.2 Tests added or updated

test_integrity_fusion_policy

- from_string/to_string fusion modes
- gnss_only final result GNSS source

- `lidar_only` final result `lidar` source
- `fallback_only` final result `fallback` source
- `max_pl` valid enabled sources `per-axis max`
- disabled source `numerical failure/conservative output`
- invalid source `valid fusion` source `valid=false` `failure_reason`
- `require_valid_gnss=true` GNSS missing/invalid `finite conservative PL` failure reason
- `require_valid_lidar=true` LiDAR missing/invalid `finite conservative PL` failure reason
- all invalid `finite conservative HPL/VPL/PL_E/PL_N/PL_U`
- `weighted_debug_only` deterministic fallback `max_pl`

`test_araim`

```
IntegrityReportMappingTest.SourceFusionAndFailureFieldsMapToRosMessage
```

internal `IntegrityReport` `ROS iap::msg::IntegrityReport`

```
gnss_valid, lidar_valid, fallback_valid
gnss_hpl, gnss_vpl, gnss_pl_e, gnss_pl_n, gnss_pl_u
lidar_hpl, lidar_vpl, lidar_pl_e, lidar_pl_n, lidar_pl_u
fallback_hpl, fallback_vpl
fusion_mode
final_hpl_source, final_vpl_source, final_pl_source
fallback_pl_invalid, gnss_araim_invalid, lidar_integrity_invalid
hal_invalid, val_invalid, im_invalid
any_nan_rejected, any_inf_rejected
negative_variance_rejected, degenerate_geometry
failure_reason
```

11.3 Fusion mode table

| Mode / behavior | Expected | Result |
|----------------------------------|---|--------|
| <code>gnss_only</code> | Only valid GNSS source contributes; final sources are <code>GNSS</code> | PASS |
| <code>lidar_only</code> | Only valid LiDAR source contributes; final sources are <code>LIDAR</code> | PASS |
| <code>fallback_only</code> | Only valid fallback source contributes; final sources are <code>FALLBACK</code> | PASS |
| <code>max_pl</code> | Per-axis max across valid enabled sources | PASS |
| <code>weighted_debug_only</code> | Deterministic; currently equal to <code>max_pl</code> | PASS |
| disabled source | Ignored and not treated as numerical failure | PASS |
| invalid source | Ignored for valid fusion; source validity/failure remains observable | PASS |
| required GNSS missing/invalid | Finite conservative output with GNSS failure reason | PASS |
| required LiDAR missing/invalid | Finite conservative output with LiDAR failure reason | PASS |
| all sources invalid | Finite conservative output, no NaN/Inf | PASS |

11.4 Per-axis max fusion numeric result

Test fixture:

| Source | HPL | VPL | PL_E | PL_N | PL_U |
|----------|-----|-----|------|------|------|
| GNSS | 3 | 10 | 3 | 2 | 10 |
| LiDAR | 8 | 4 | 8 | 4 | 4 |
| Fallback | 5 | 5 | 5 | 5 | 5 |

Expected and observed fused result:

| Field | Expected | Result |
|---------------------------------|--------------------|--------|
| <code>PL_E</code> | 8 | PASS |
| <code>PL_N</code> | 5 | PASS |
| <code>PL_U</code> | 10 | PASS |
| <code>HPL=max(PL_E,PL_N)</code> | 8 | PASS |
| <code>VPL=PL_U</code> | 10 | PASS |
| <code>final_hpl_source</code> | <code>LIDAR</code> | PASS |

| Field | Expected | Result |
|------------------|----------|--------|
| final_vpl_source | GNSS | PASS |
| final_pl_source | GNSS | PASS |

11.5 Disabled vs invalid source result

Disabled source case:

- GNSS disabled source was assigned very large sentinel-like PL values.
- Fusion ignored it because `enabled=false`.
- Result used LiDAR for HPL and fallback for VPL.
- `failure_reason` stayed empty and `any_source_valid=true`

Invalid source case:

- LiDAR source was constructed with `valid=false` and `failure_reason="LiDAR ARAIM result invalid"`
- Fusion ignored invalid LiDAR for valid fused PL.
- Result used fallback for HPL and GNSS for VPL.
- Invalid source diagnostics remained observable through the source object fields.

11.6 Required-source missing behavior

| Required source | Missing/invalid input | Output | Failure reason |
|-----------------|---|---|--|
| GNSS | <code>require_valid_gnss=true</code> , GNSS invalid | finite conservative
HPL/VPL/PL_E/PL_N/PL_U | required GNSS source missing or invalid |
| LiDAR | <code>require_valid_lidar=true</code> , LiDAR invalid | finite conservative
HPL/VPL/PL_E/PL_N/PL_U | required LiDAR source missing or invalid |
| All sources | fallback/GNSS/LiDAR invalid | finite conservative
HPL/VPL/PL_E/PL_N/PL_U | no valid integrity source available |

required-source failure all-invalid failure finite conservative PL NaN/Inf

11.7 ROS message mapping result

`fill_integrity_report_msg(report, msg)` source/fusion/failure fields

| Mapping group | Result |
|--|--------|
| GNSS validity and H/V/per-axis PL | PASS |
| LiDAR validity and H/V/per-axis PL | PASS |
| Fallback validity and H/V PL | PASS |
| <code>fusion_mode</code> | PASS |
| <code>final_hpl_source/final_vpl_source/final_pl_source</code> | PASS |
| numerical failure flags | PASS |
| <code>failure_reason</code> | PASS |

API limitation `IntegrityReport.msg` per-source failure reason `failure_reason` per-source valid/failure flags `fallback_valid` internal field `!report.numerical_failure.fallback_pl_invalid`

11.8 Build/test results

```
colcon build --symlink-install --packages-select iap \
--cmake-args -DCMAKE_BUILD_TYPE=RelWithDebInfo -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
```

```
Summary: 1 package finished [10.8s]
```

`stderr` deprecated API warnings `predict_geometry` deprecated `Araim` alias fusion/report mapping tests

```
colcon test --packages-select iap \
  --ctest-args -R "test_integrity_fusion_policy|test_araim" --output-on-failure
```

📄

```
Summary: 1 package finished [0.21s]
```

📄📄📄📄📄

```
test_integrity_fusion_policy: 15 tests, 0 failures, 0 errors
test_araim: 80 tests, 0 failures, 0 errors
```

colcon test-result --all📄

```
Summary: 596 tests, 3 errors, 396 failures, 15 skipped
```

📄📄📄📄📄📄📄📄 bspline_opt📄ego_planner📄path_searching 📄 iap 📄📄📄📄📄📄iap 📄📄📄📄 0 failures / 0 errors📄

11.9 Previous ARAIM and H/V tests

- test_araim 📄📄80 tests, 0 failures, 0 errors📄
- 📄 GNSS ARAIM golden-output📄degenerate geometry📄fault injection tests 📄📄📄 test_araim 📄📄📄📄
- 📄 H/V safety semantics tests 📄📄📄 test_araim 📄📄📄📄
- test_integrity_fusion_policy 📄📄15 tests, 0 failures, 0 errors📄

11.10 Final conclusion

- 📄 integrity subsystem 📄📄📄 GNSS📄LiDAR 📄 fallback source 📄 enabled/valid/failure 📄📄
- IntegrityFusionPolicy 📄 gnss_only📄lidar_only📄fallback_only📄max_pl 📄 weighted_debug_only 📄 fusion semantics 📄📄 unit validation📄
- max_pl 📄 per-axis max 📄 final source attribution 📄📄 HPL/VPL dominating source📄
- disabled source 📄📄 numerical failure📄invalid source 📄📄 valid fusion📄📄 diagnostics 📄📄📄
- required source 📄📄📄 conservative finite PL📄
- ROS message mapping 📄📄 source/fusion/failure diagnostics📄
- 📄📄📄📄 fusion policy 📄 report mapping helper 📄📄📄📄

12. LiDAR Integrity Evaluator Validation📄Synthetic FIM monotonicity

📄📄📄📄 LiDAR integrity evaluator 📄 synthetic information matrix 📄📄📄📄📄 GNSS ARAIM 📄📄📄📄 fusion policy📄📄 planner📄📄📄📄📄

📄📄📄📄📄 **LiDAR Integrity Evaluator Validation**📄📄📄📄 LidarAraim::run() 📄 synthetic FIM 📄📄 pose covariance📄📄📄📄📄📄
evaluateLidarFim() API📄

12.1 Files modified

| File | Change |
|--------------------------------------|---|
| src/iap/test/test_araim.cpp | 📄 test-local synthetic FIM adapter 📄 LidarIntegrityValidationTest 3 📄📄📄 |
| src/iap/docs/dev_ARAIM/ARAIM_test.md | 📄📄📄📄📄📄 |

📄📄📄 GNSS ARAIM formulas📄fusion policy semantics📄planner behavior📄CMake target structure📄Araim 📄📄 inheritance📄

12.2 Existing API and injection method

📄 monitor-side LiDAR integrity evaluator 📄

```
LidarAraim::run(const LidarAraimSnapshot& snapshot,
               const FG0PositionInfo& fgo_info)
```

📄 API 📄 pose covariance 📄 LiDAR blocks📄📄📄 3x3 raw FIM📄📄📄📄 test-local adapter📄

- positive-definite 3x3 FIM 📄 cov_pos = FIM.inverse() 📄📄 FG0PositionInfo::pose_cov_6x6 📄 LidarAraimSnapshot::pose_cov_6x6 📄 position block📄
- snapshot 📄📄📄 zero-information LiDAR block📄📄📄 LidarAraim::run() 📄 hypothesis path📄📄📄 synthetic FIM📄
- singular FIM 📄📄📄 test-local adapter 📄📄 invalid/conservative finite result📄📄 failure_reason="singular_lidar_fim"📄

📄📄 synthetic point cloud generation📄

12.3 Tests added

□□□□□

```
LidarIntegrityValidationTest.FimMonotonicity
LidarIntegrityValidationTest.DegenerateFimProducesLargeDirectionalPlOrInvalid
LidarIntegrityValidationTest.SingularFimIsValidOrConservativeFinite
```

□□□□□

- stronger FIM □□□□□□□ PL□
- degenerate FIM □□□□□ PL □□□□□□□□□ invalid/degraded□
- singular FIM □□□ NaN/Inf□□ invalid/conservative finite□

12.4 Synthetic FIM inputs

| Case | FIM |
|------------|--------------------|
| Strong | diag(100,100,100) |
| Weak | diag(25,25,25) |
| Degenerate | diag(100,1e-6,100) |
| Singular | diag(100,0,100) |

12.5 Recorded metrics

| Case | lidar_valid | HPL | VPL | PL_E | PL_N | PL_U | condition_number | lidar_worst_mode |
|------------|-------------|------------|------------|------------|------------|------------|------------------|------------------|
| Strong | 1 | 0.7 | 0.7 | 0.7 | 0.7 | 0.7 | 1 | FAULT_FREE |
| Weak | 1 | 1.4 | 1.4 | 1.4 | 1.4 | 1.4 | 1 | FAULT_FREE |
| Degenerate | 1 | 7000 | 0.7 | 0.7 | 7000 | 0.7 | 100000000 | H_LEVEL(0) |
| Singular | 0 | 1000000000 | 1000000000 | 1000000000 | 1000000000 | 1000000000 | 1000000000000 | NONE |

12.6 Pass/fail table

| Requirement | Observed | Result |
|---|--|--------|
| PL(strong) <= PL(weak) | strong HPL/VPL 0.7, weak HPL/VPL 1.4 | PASS |
| Per-axis PL does not increase under stronger information | strong PL_E/N/U 0.7, weak PL_E/N/U 1.4 | PASS |
| Degenerate direction produces larger directional PL or invalid/degraded | degenerate PL_N 7000, PL_E/PL_U 0.7, condition 1e8 | PASS |
| Singular FIM does not produce NaN | singular HPL/VPL/PL_E/PL_N/PL_U finite | PASS |
| Singular FIM does not produce Inf | singular HPL/VPL/PL_E/PL_N/PL_U finite | PASS |
| Singular FIM invalid/conservative with failure reason | lidar_valid=0, conservative 1e9, reason singular_lidar_fim | PASS |

12.7 Build/test results

□□□□□

```
colcon build --symlink-install --packages-select iap \
--cmake-args -DCMAKE_BUILD_TYPE=RelWithDebInfo -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
```

□□□

```
Summary: 1 package finished [11.5s]
```

□□□stderr □□□□□ deprecated API warnings□□□ predict_geometry □ deprecated Araith alias□□□□ LiDAR integrity evaluator tests □□□

□□□□□□□□□

```
./build/iap/test_araim --gtest_filter='LidarIntegrityValidationTest.*' --gtest_brief=1
```



```

```
[=====] 3 tests from 1 test suite ran. (0 ms total)
[PASSED] 3 tests.
```

```

```
colcon test --packages-select iap \
  --ctest-args -R "test_araim" --output-on-failure
```

```

```
Summary: 1 package finished [0.17s]
```

```

```
test_araim: 83 tests, 0 failures, 0 errors
LidarIntegrityValidationTest: 3 tests, 0 failures, 0 errors
```

colcon test-result --all

```
Summary: 598 tests, 3 errors, 396 failures, 15 skipped
```

bspline_opt ego_planner path_searching iap 0 failures / 0 errors

12.8 Limitations

- direct public raw-3x3-FIM LiDAR PL API evaluateLidarFim(Eigen::Matrix3d)
- LidarAraimResult failure_reason failure reason test-local synthetic FIM adapter
- LidarAraimResult condition number input FIM condition number subset diagnostics
- singular raw FIM covariance singular case test-local adapter invalid/conservative finite

12.9 Final conclusion

- Synthetic FIM monotonicity LiDAR PL
- directional PL PL_N=7000 condition number 1e8
- singular FIM NaN/Inf invalid/conservative finite
- LiDAR production evaluator

13. LiDAR Integrity Evaluator Validation Geometry observability trend

LiDAR integrity evaluator synthetic FIM GNSS ARAIM fusion policy planner production evaluator

13.1 Files modified

| File | Change |
|--------------------------------------|---|
| src/iap/test/test_araim.cpp | LidarIntegrityValidationTest.GeometryObservabilityTrend |
| src/iap/docs/dev_ARAIM/ARAIM_test.md | |

12 test-local synthetic FIM adapter positive-definite 3x3 FIM cov_pos = FIM.inverse() FG0PositionInfo::pose_cov_6x6 LidarAraimSnapshot::pose_cov_6x6 position block

13.2 Synthetic geometry inputs

| Geometry | FIM | Intuition |
|--------------|------------------|-------------|
| feature-rich | diag(100,100,80) | |
| corridor | diag(5,100,60) | x/E y/N z/U |
| sparse | diag(1,1,1) | |

synthetic point cloud generation

13.3 Test added

```

```
LidarIntegrityValidationTest.GeometryObservabilityTrend
```

~~~~~

- feature-rich `lidar_valid` `finite`
- sparse `finite` `valid` `degraded` `invalid`
- `feature.HPL < corridor.HPL`
- sparse valid `corridor.HPL < sparse.HPL` failure reason
- corridor `PL` `corridor.PL_E > feature.PL_E`
- corridor condition number `feature-rich` condition number
- corridor `lidar_worst_mode`

13.4 Recorded metrics

Geometry	lidar_valid	HPL	VPL	PL_E	PL_N	PL_U	condition_number	lidar_worst_mode
feature-rich	1	0.7	0.7826237921	0.7	0.7	0.7826237921	1.25	H_LEVEL(0)
corridor	1	3.1304951685	0.9036961141	3.1304951685	0.7	0.9036961141	20	H_LEVEL(0)
sparse	1	7	7	7	7	7	1	H_LEVEL(0)

13.5 Pass/fail table

Requirement	Observed	Result
feature-rich LiDAR PL smallest	feature HPL 0.7 < corridor HPL 3.1304951685	PASS
corridor PL smaller than sparse when sparse valid	corridor HPL 3.1304951685 < sparse HPL 7	PASS
corridor weak direction PL grows	corridor PL_E 3.1304951685 > feature PL_E 0.7	PASS
corridor condition exceeds feature condition	corridor condition 20 > feature condition 1.25	PASS
all HPL/VPL finite	feature/corridor/sparse HPL/VPL all finite	PASS
worst mode observable	corridor lidar_worst_mode=H_LEVEL(0)	PASS

13.6 Build/test results

~~~~~

```
colcon build --symlink-install --packages-select iap \
 --cmake-args -DCMAKE_BUILD_TYPE=RelWithDebInfo -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
```

~~~

```
Summary: 1 package finished [11.5s]
```

~~~~~stderr~~~~~ deprecated API warnings~~~~~ predict\_geometry deprecated ARAIM alias~~~~~ LiDAR geometry observability tests~~~~~  
~~~~~ LiDAR validation tests~~~~~

```
./build/iap/test_araim --gtest_filter='LidarIntegrityValidationTest.*' --gtest_brief=1
```

~~~~~

```
[=====] 4 tests from 1 test suite ran. (0 ms total)
[PASSED] 4 tests.
```

~~~~~

```
colcon test --packages-select iap \
 --ctest-args -R "test_araim" --output-on-failure
```

~~~~~

```
Summary: 1 package finished [0.16s]
```

~~~~~

```
test_araim: 84 tests, 0 failures, 0 errors
LidarIntegrityValidationTest: 4 tests, 0 failures, 0 errors
```

colcon test-result --all

```
Summary: 599 tests, 3 errors, 396 failures, 15 skipped
```

~~~~~ bspline\_opt ego\_planner path\_searching ~~~~ iap ~~~~~~ iap ~~~~~~ 0 failures / 0 errors

13.7 Limitations

- synthetic FIM
- direct public raw-3x3-FIM LiDAR PL API test-local FIM-to-covariance adapter
- LidarAraimResult failure\_reason condition number test-local derived metrics
- sparse diag(1,1,1) valid finite PL invalid/degraded sparse output failure reason

13.8 Final conclusion

- Geometry observability trend feature-rich PL < corridor PL < sparse PL
- corridor case x/E PL\_E HPL
- sparse case finite HPL/VPL NaN/Inf
- production LiDAR evaluator

14. LiDAR Integrity Evaluator ValidationBlock fault injection

LiDAR block-level integrity path “LiDAR/VGICP block” production evaluator GNSS single-satellite ARAIM excluded block list

14.1 Block-level API confirmation

block-level LiDAR integrity API

| API / field                         | Location                              | Meaning                                 |
|-------------------------------------|---------------------------------------|-----------------------------------------|
| LidarAraimBlock                     | include/iap/integrity/lidar_araim.hpp | LiDAR/VGICP block metadata              |
| LidarAraimSnapshot::blocks          | include/iap/integrity/lidar_araim.hpp | frame LiDAR block list                  |
| LidarAraimBlock::Lambda_B           | include/iap/integrity/lidar_araim.hpp | block information matrix                |
| LidarAraimBlock::eta_B              | include/iap/integrity/lidar_araim.hpp | block Hessian linear term residual-like |
| LidarHypothesis::block_indices      | include/iap/integrity/lidar_araim.hpp | hypothesis block indices                |
| LidarSubsetSolution::fault_detected | include/iap/integrity/lidar_araim.hpp | subset-level fault detection flag       |

runtime VGICP population

- CPU path IntegratedVGICPFactor linearization Lambda\_B eta\_B
- GPU path IntegratedVGICPFactorGPU linearization Lambda\_B eta\_B

block\_residual API residual-like signal eta\_B

14.2 Files modified

| File                                 | Change                                                                                          |
|--------------------------------------|-------------------------------------------------------------------------------------------------|
| src/iap/test/test_araim.cpp          | test-local block fault helpers LidarIntegrityValidationTest.BlockFaultInjectionBadBlockDetected |
| src/iap/docs/dev_ARAIM/ARAIM_test.md |                                                                                                 |

GNSS ARAIM formulasfusion policy semanticsplanner behaviorCMake target structureAraim inheritance

14.3 Test construction

5 synthetic LiDAR blocks

| Block | target_frame_id | level_id | eta_B(3) | Meaning                   |
|-------|-----------------|----------|----------|---------------------------|
| 1     | 1               | 1        | 0.1      | good residual-like signal |
| 2     | 2               | 2        | 0.1      | good residual-like signal |
| 3     | 3               | 3        | 10.0     | bad residual-like signal  |
| 4     | 4               | 4        | 0.1      | good residual-like signal |
| 5     | 5               | 5        | 0.1      | good residual-like signal |

block moderate positive  $\Lambda_B = I$  subset downdate finite LidarAraimResult top-level worst\_block\_id result.hypotheses result.subsets single-block target/level hypotheses worst block

```
LidarIntegrityValidationTest.BlockFaultInjectionBadBlockDetected
```

14.4 Recorded metrics

| Metric                   | Value               |
|--------------------------|---------------------|
| lidar_valid              | 1                   |
| lidar_hpl                | 4.6865518505672252  |
| lidar_vpl                | 2.6065518505672256  |
| lidar_pl_e               | 4.6865518505672252  |
| lidar_pl_n               | 2.6065518505672256  |
| lidar_pl_u               | 2.6065518505672256  |
| n_hypotheses             | 11                  |
| n_detected               | 3                   |
| lidar_worst_mode         | H_LEVEL(3)          |
| derived worst_block_id   | 3                   |
| derived worst_block_mode | H_TARGET(3)         |
| bad block d_E            | -1.1111111111111112 |
| bad block T_E            | 0.42163702135578368 |
| bad block PL_E           | 2.532748132466895   |
| bad block fault_detected | 1                   |
| clean baseline HPL       | 2.7065518505672257  |
| clean baseline VPL       | 2.6065518505672256  |

14.5 Pass/fail table

| Requirement                                | Observed                                                  | Result |
|--------------------------------------------|-----------------------------------------------------------|--------|
| HPL/VPL/per-axis PL finite                 | all finite                                                | PASS   |
| LiDAR output valid or conservative invalid | lidar_valid=1                                             | PASS   |
| bad block produces detection               | n_detected=3                                              | PASS   |
| block 3 single-block hypothesis detected   | fault_detected=1, abs(d_E)=1.111111111 > T_E=0.4216370214 | PASS   |
| derived worst block is block 3             | worst_block_id=3                                          | PASS   |
| injected PL larger than clean baseline     | HPL 4.6865518506 > clean HPL 2.7065518506                 | PASS   |
| worst mode observable                      | lidar_worst_mode=H_LEVEL(3)                               | PASS   |

14.6 Build/test results

```
colcon build --symlink-install --packages-select iap \
--cmake-args -DCMAKE_BUILD_TYPE=RelWithDebInfo -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
```

```


```

```
Summary: 1 package finished [11.6s]
```

```
std::errc deprecated API warnings predict_geometry deprecated ARAIM alias LiDAR block fault injection test
```

LiDAR-focused tests

```
./build/iap/test_araim \
--gtest_filter='LidarIntegrityValidationTest.*:LidarArAIMTest.*Block*' \
--gtest_brief=1
```

```


```

```
[=====] 7 tests from 2 test suites ran. (0 ms total)
[PASSED] 7 tests.
```

```


```

```
colcon test --packages-select iap \
--ctest-args -R "test_araim" --output-on-failure
```

```


```

```
Summary: 1 package finished [0.17s]
```

```


```

```
test_araim: 85 tests, 0 failures, 0 errors
LidarIntegrityValidationTest: 5 tests, 0 failures, 0 errors
```

```
colcon test-result --all
```

```
Summary: 600 tests, 3 errors, 396 failures, 15 skipped
```

```
bspline_opt ego_planner path_searching iap 0 failures / 0 errors
```

## 14.7 Limitations

- top-level `worst_block_id` single-block target/level hypotheses
- excluded-block output block exclusion fault detection worst block observability
- `LidarArAIMResult` `failure_reason`
- `worst_mode` hypothesis label `H_LEVEL(3) / H_TARGET(3)` literal `high_residual` string

## 14.8 Final conclusion

- LiDAR block-level API block fault injection unit validation
- block 3 bad residual-like signal LiDAR integrity evaluator finite PL fault hypothesis/subset diagnostics worst block 3
- bad block injection HPL clean baseline 2.7065518506 4.6865518506
- production LiDAR evaluator

## 14.9 Mathematical principle

VGICP/LiDAR block block

$$J_B(\delta x) \approx 0.5 * \delta x^T \Lambda_B \delta x + \eta_B^T \delta x$$

```


```

- `Lambda_B` block 6D pose increment Hessian / information matrix
- `eta_B` block Hessian linear term residual-like signal
- fault `eta_B(3)` E / x position component

block fault-free information

```
Lambda_0 = Sigma_0^{-1}
```

$\Sigma_0$  pose covarianceLiDAR evaluator fault hypothesis subset downdate block fault-free information block

```
Lambda_f = Lambda_0 - sum(Lambda_B)
eta_f = -sum(eta_B)
```

hypothesis separation / displacement

```
delta_f = Lambda_f^{-1} eta_f
d_E = delta_f(E)
d_N = delta_f(N)
d_U = delta_f(U)
```

block  $\eta_B(3)$  block single-block hypothesis hypothesis  $|d_E|$  block 3

```
eta_B(3) = 10.0
```

good blocks

```
eta_B(3) = 0.1
```

block 3 single-block hypothesis E separation

```
d_E = -1.1111111111111112
```

fault detection separation false-alarm threshold

```
T_E = K_fa * sigma_ss_E
fault_detected = |d_E| > T_E
```

```
|d_E| = 1.1111111111111112
T_E = 0.42163702135578368
```

```
|d_E| > T_E -> fault_detected = true
```

Protection Level separationdetection thresholdmissed-detection term risk bias

```
PL_E = |d_E| + K_fa * sigma_ss_E + K_md * sigma_k_E + bias_H
PL_N = |d_N| + K_fa * sigma_ss_N + K_md * sigma_k_N + bias_H
PL_U = |d_U| + K_fa * sigma_ss_U + K_md * sigma_k_U + bias_V
HPL = max(PL_E, PL_N)
VPL = PL_U
```

injected bad block E  $PL_E$

```
lidar_pl_e = 4.6865518505672252
lidar_pl_n = 2.6065518505672256
lidar_hpl = 4.6865518505672252
```

clean baseline    block   residual-like signal    HPL   

```
clean HPL = 2.7065518505672257
fault-injected HPL = 4.6865518505672252
```

block 3   eta\_B(3)   subset downdate   E-direction separation   |d\_E|   threshold   PL\_E/HPL   block-level LiDAR integrity   

15. LiDAR Integrity Evaluator Validation: Point Cloud to FIM to LiDAR PL

15.1 Experiment Scope

```
synthetic point cloud -> normal/PCA primitives -> LiDAR FIM -> LiDAR PL
```

LiDAR Integrity Evaluator Validation    /PCA   advisory FIM   FIM-to-LidarAraim adapter   LiDAR PL   GNSS ARAIM   fusion policy   certified real-world LiDAR ARAIM   

15.2 Files Added Or Updated

- Added src/iap/scripts/araim\_validation/generate\_synthetic\_lidar\_clouds.py.
- Updated src/iap/test/test\_araim.cpp.
- Appended this report section to src/iap/docs/dev\_ARAIM/ARAIM\_test.md.
- Generated deterministic fixtures in src/iap/results/araim\_validation/:
  - feature\_rich\_cloud.csv
  - corridor\_cloud.csv
  - sparse\_cloud.csv
  - lidar\_pointcloud\_fim\_pl\_metrics.csv
  - lidar\_block\_fault\_metrics.csv
  - feature\_rich\_cloud.png
  - corridor\_cloud.png
  - sparse\_cloud.png
  - lidar\_cloud\_inputs\_comparison.png
  - lidar\_fim\_eigenvalue\_spectrum.png
  - lidar\_fim\_condition\_number.png
  - lidar\_directional\_pl.png
  - lidar\_hpl\_vpl\_comparison.png
  - lidar\_condition\_vs\_pl.png
  - lidar\_fim\_heatmaps.png
  - lidar\_block\_fault\_hpl\_vpl.png
  - lidar\_block\_fault\_detection\_margin.png

15.3 Test Added

Added:

```
LidarIntegrityValidationTest.PointCloudToFimToPLTrend
```

The test:

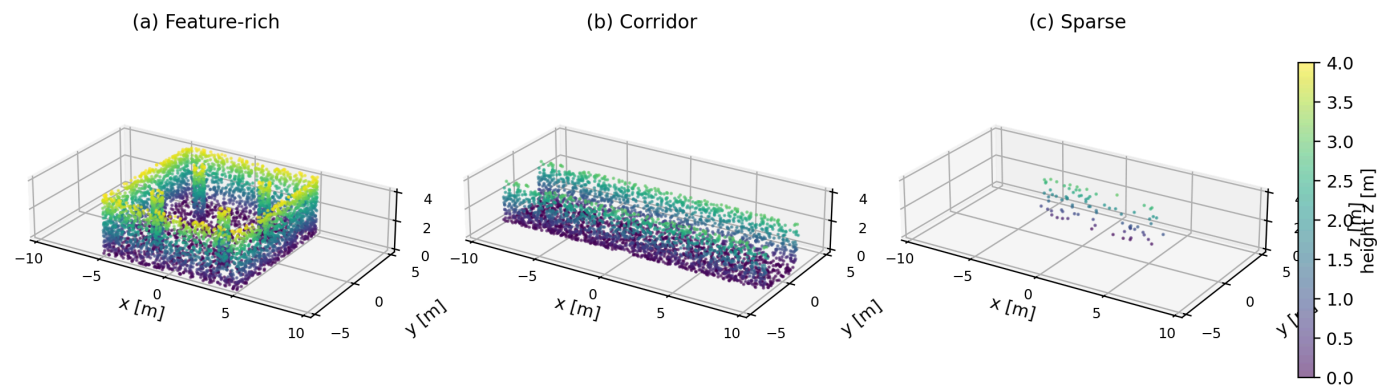
- Loads generated CSV point clouds.
- Converts points to PCA/normal primitives with make\_lidar\_fim\_primitives().
- Computes advisory LiDAR FIM with LidarObservabilityFim::evaluate\_advisory\_fim().
- Converts positive-definite 3x3 FIM to covariance through the existing test-local synthetic-FIM adapter.
- Runs the current LidarAraim evaluator and records PL metrics.

15.4 Synthetic Clouds

| Case         | Geometry                           | Points | Expected behavior                                    |
|--------------|------------------------------------|--------|------------------------------------------------------|
| feature-rich | x=+/-5, y=+/-5, floor z=0, columns | 5380   | rich normal directions, low condition, low PL        |
| corridor     | parallel walls y=+/-2, floor z=0   | 3600   | weak x/E observability, higher condition, larger HPL |
| sparse       | small single-wall cloud            | 80     | degenerate or weak FIM, large PL or invalid/degraded |

15.4.1 Point Cloud Visualizations

The normalized input visualization uses a shared camera angle, shared axis range, common tick style, and height-based color. This makes the geometry contrast visible without changing plotting scale between scenes.



15.5 Measured Results

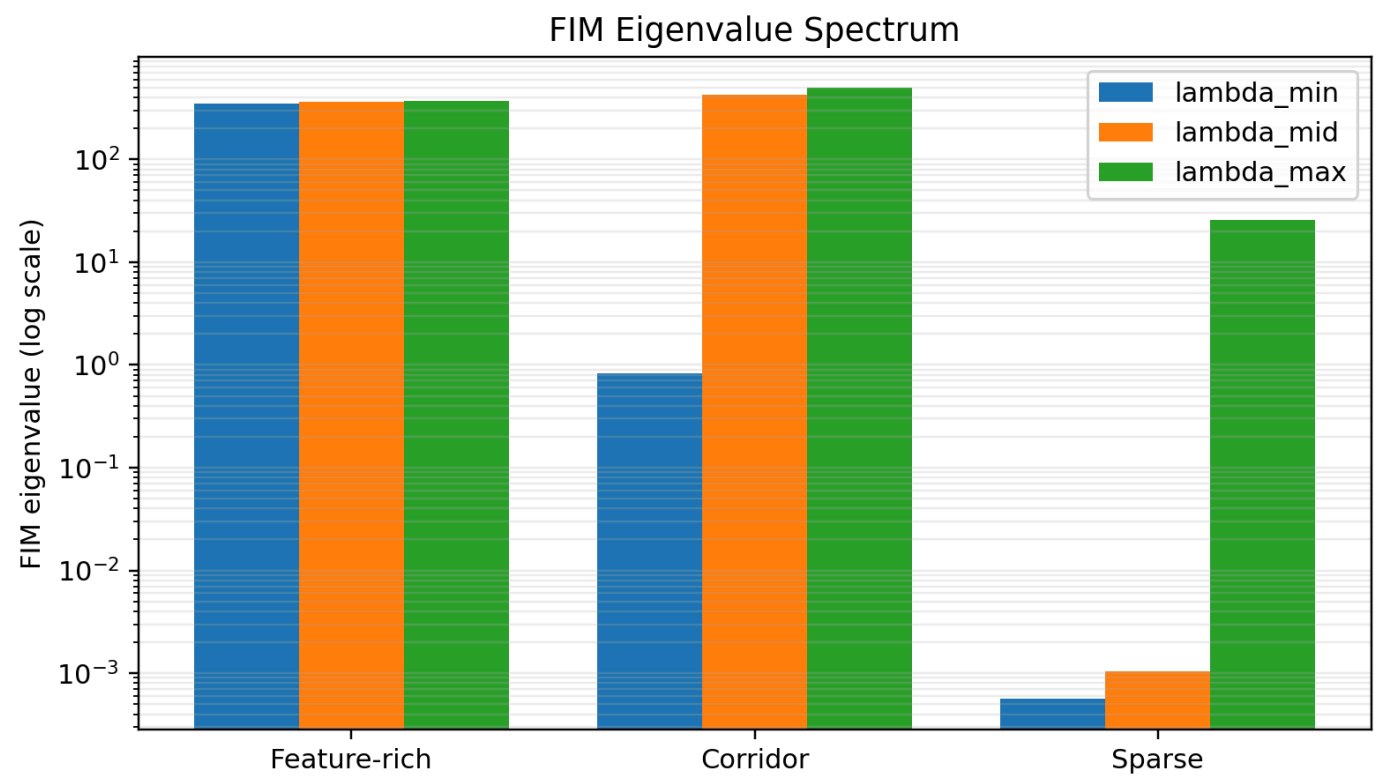
| Case         | primitives | valid normals | FIM valid | FIM trace   | min eig    | max eig    | condition    | LiDAR valid | HPL        | VPL        |
|--------------|------------|---------------|-----------|-------------|------------|------------|--------------|-------------|------------|------------|
| feature-rich | 2014       | 2014          | 1         | 1077.999806 | 347.282347 | 368.059328 | 1.059827     | 1           | 0.368296   | 0.374769   |
| corridor     | 1653       | 1653          | 1         | 927.484564  | 0.826432   | 498.590389 | 603.304732   | 1           | 7.700064   | 0.338277   |
| sparse       | 58         | 58            | 1         | 25.528370   | 0.000566   | 25.526766  | 45123.042177 | 1           | 230.306337 | 284.228246 |

The table values are also saved to:

```
src/iap/results/araim_validation/lidar_pointcloud_fim_pl_metrics.csv
src/iap/results/araim_validation/lidar_block_fault_metrics.csv
```

15.5.1 Quantitative Figures

FIM eigenvalue spectrum:

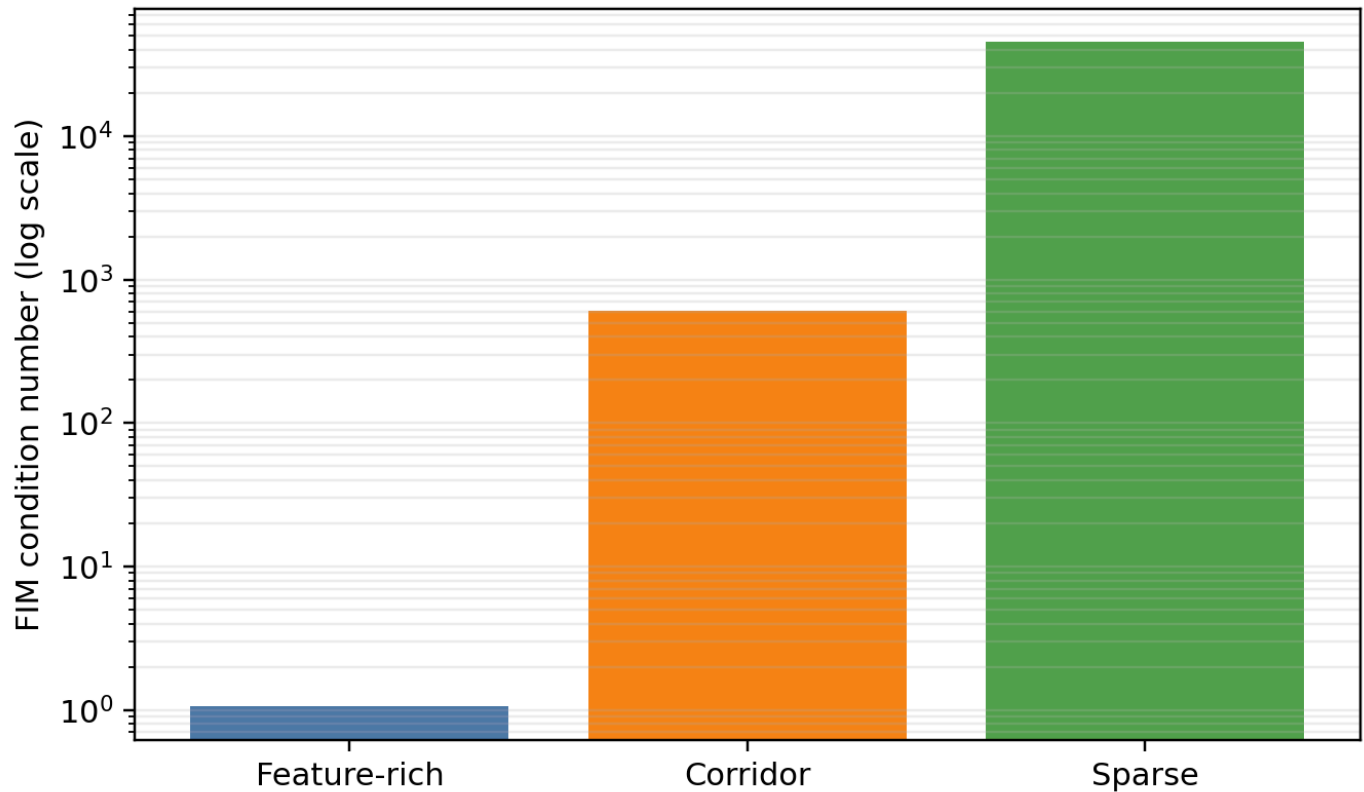


This plot shows that feature-rich geometry has three similar eigenvalues, corridor geometry has a very small minimum eigenvalue, and sparse geometry has a near-zero minimum eigenvalue. It directly explains isotropic vs degenerate LiDAR information.

FIM condition number:



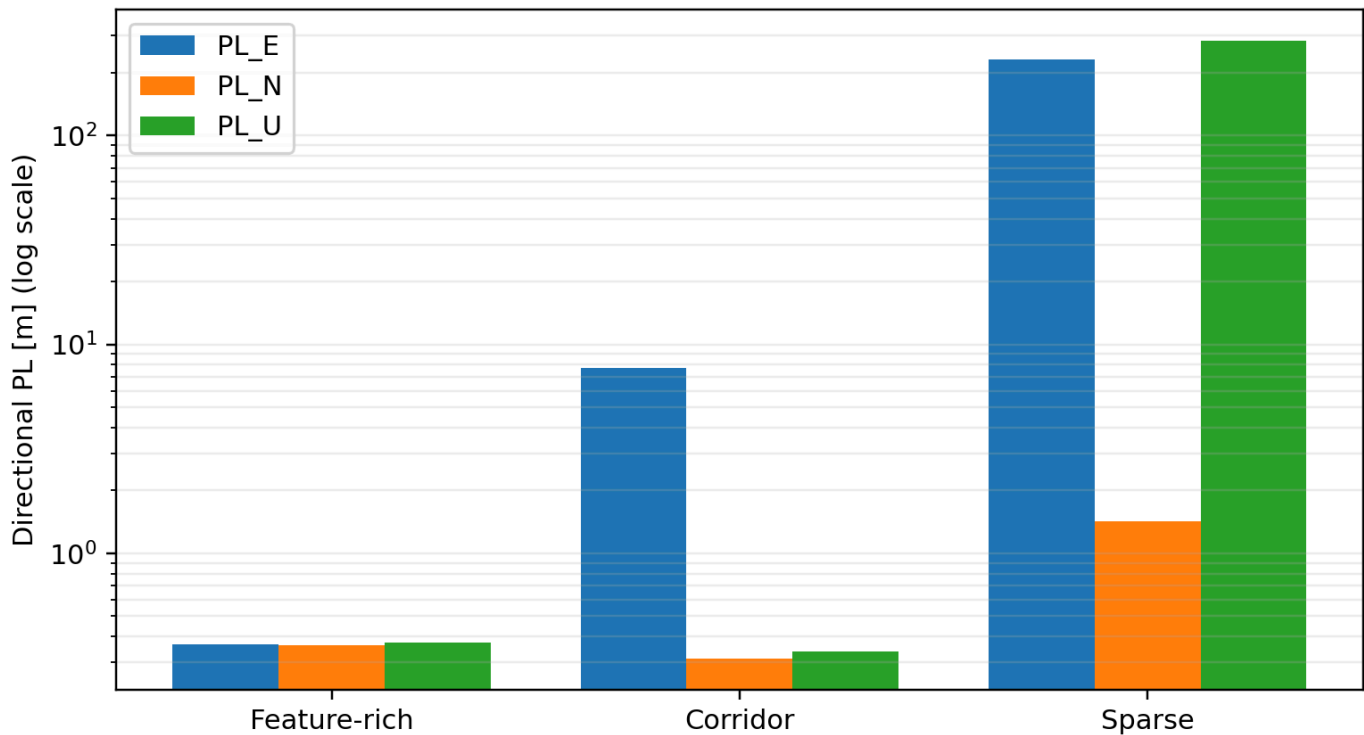
## Geometry Degeneracy From FIM Condition Number



The condition number quantifies geometry degradation: feature-rich is near 1, corridor rises to about 603, and sparse rises to about 45123.

Directional PL:

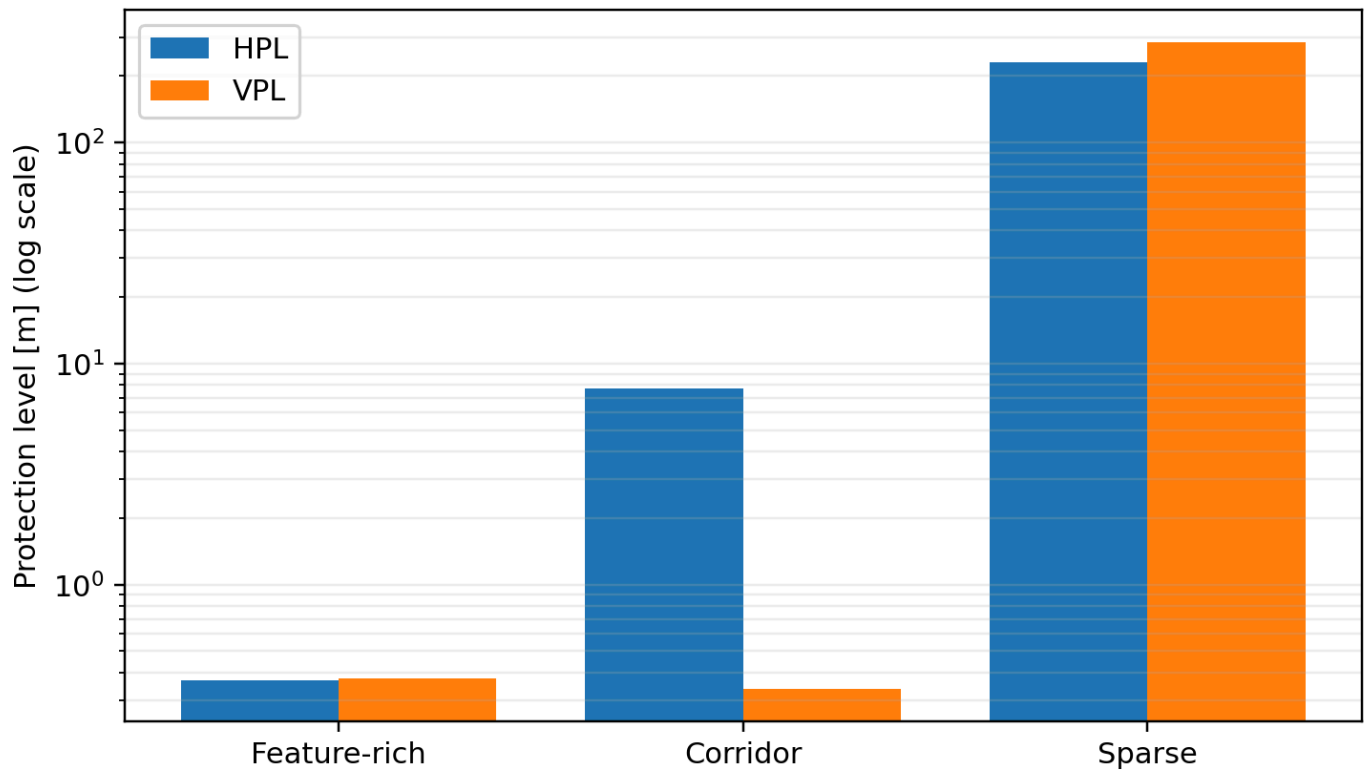
## Directional LiDAR Protection Levels



Directional PL explains the integrity effect by axis. In the corridor case, PL\_E dominates HPL because the corridor geometry weakens E-direction observability. In the sparse case, both PL\_E and PL\_U become very large.

HPL / VPL comparison:

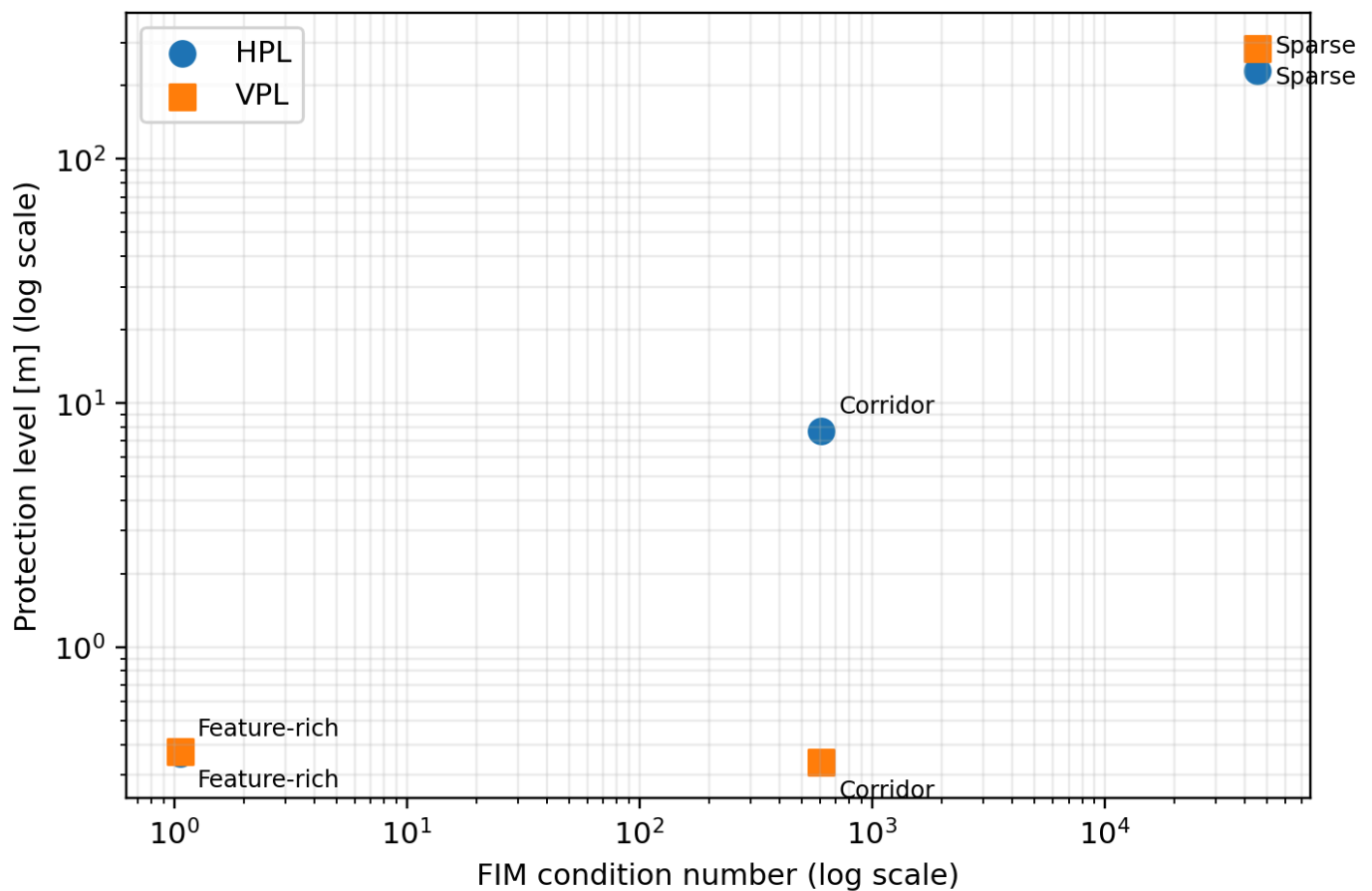
## Final LiDAR HPL / VPL



This is the final LiDAR source output summary. HPL/VPL increase as geometric observability degrades from feature-rich to corridor to sparse.

Condition number vs PL:

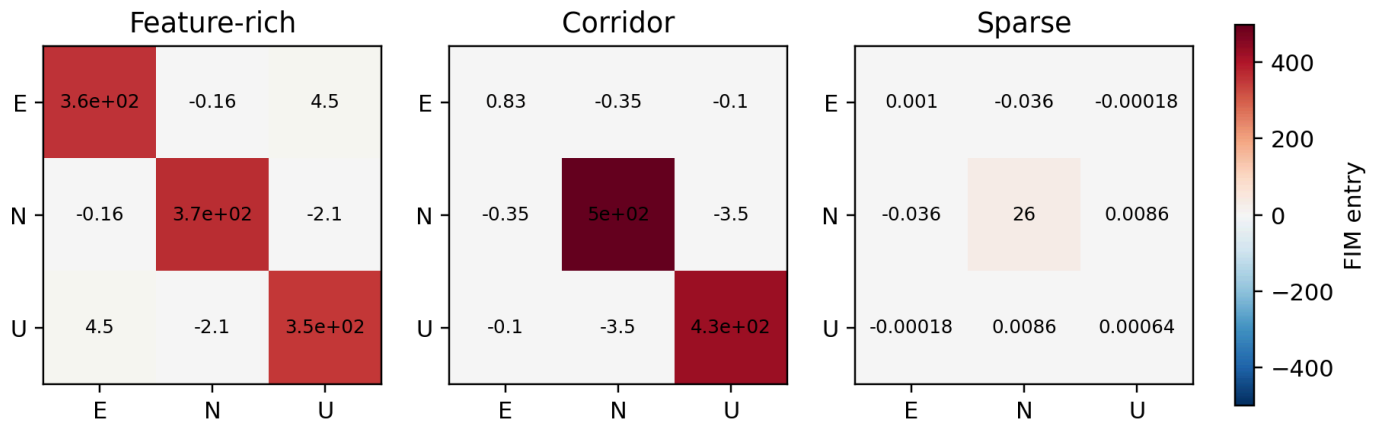
## FIM Condition Number vs LiDAR PL



This scatter plot links the FIM degeneracy metric to integrity risk: larger condition number corresponds to larger protection levels.

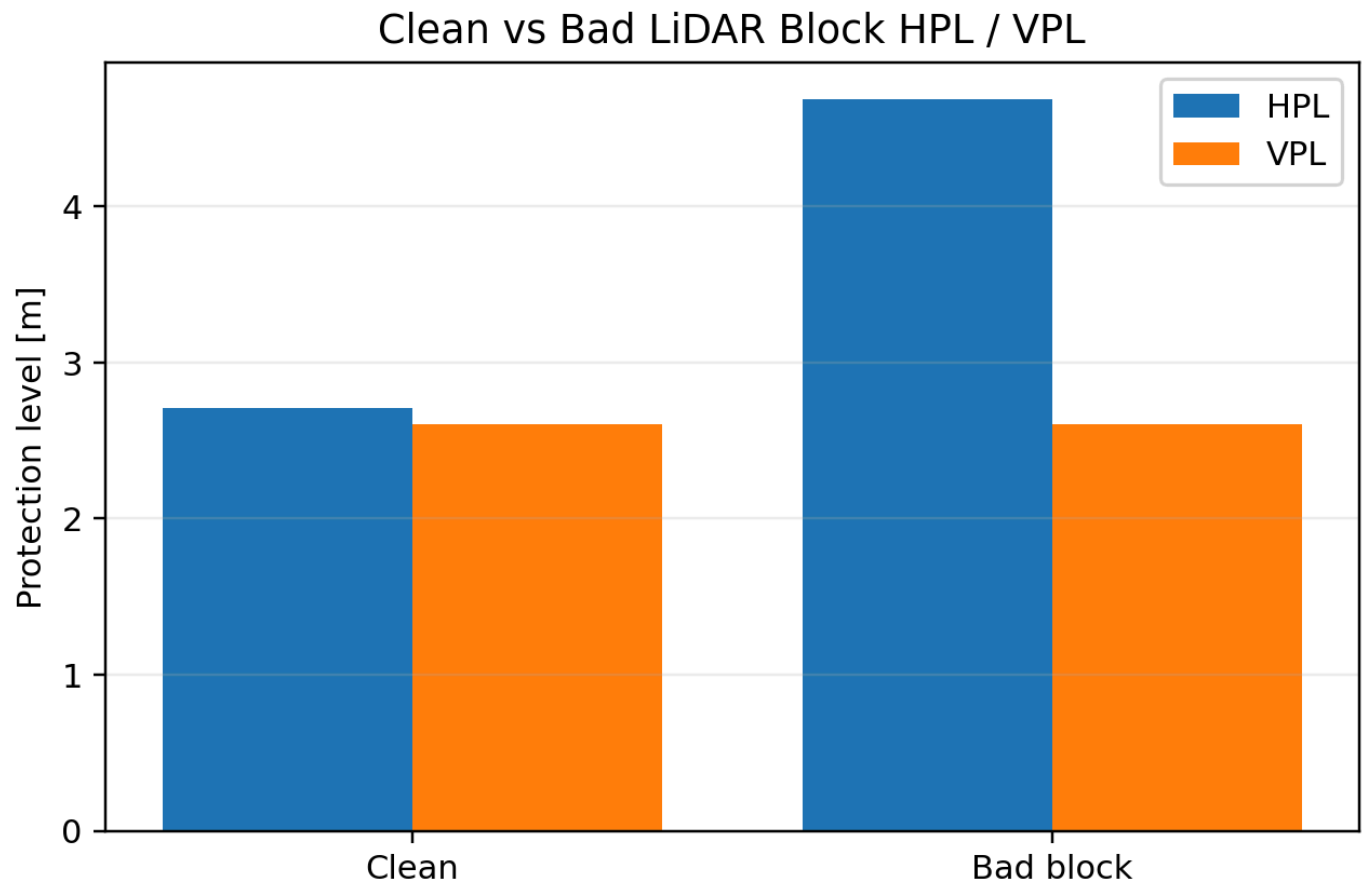
FIM heatmaps:

## LiDAR FIM Heatmaps



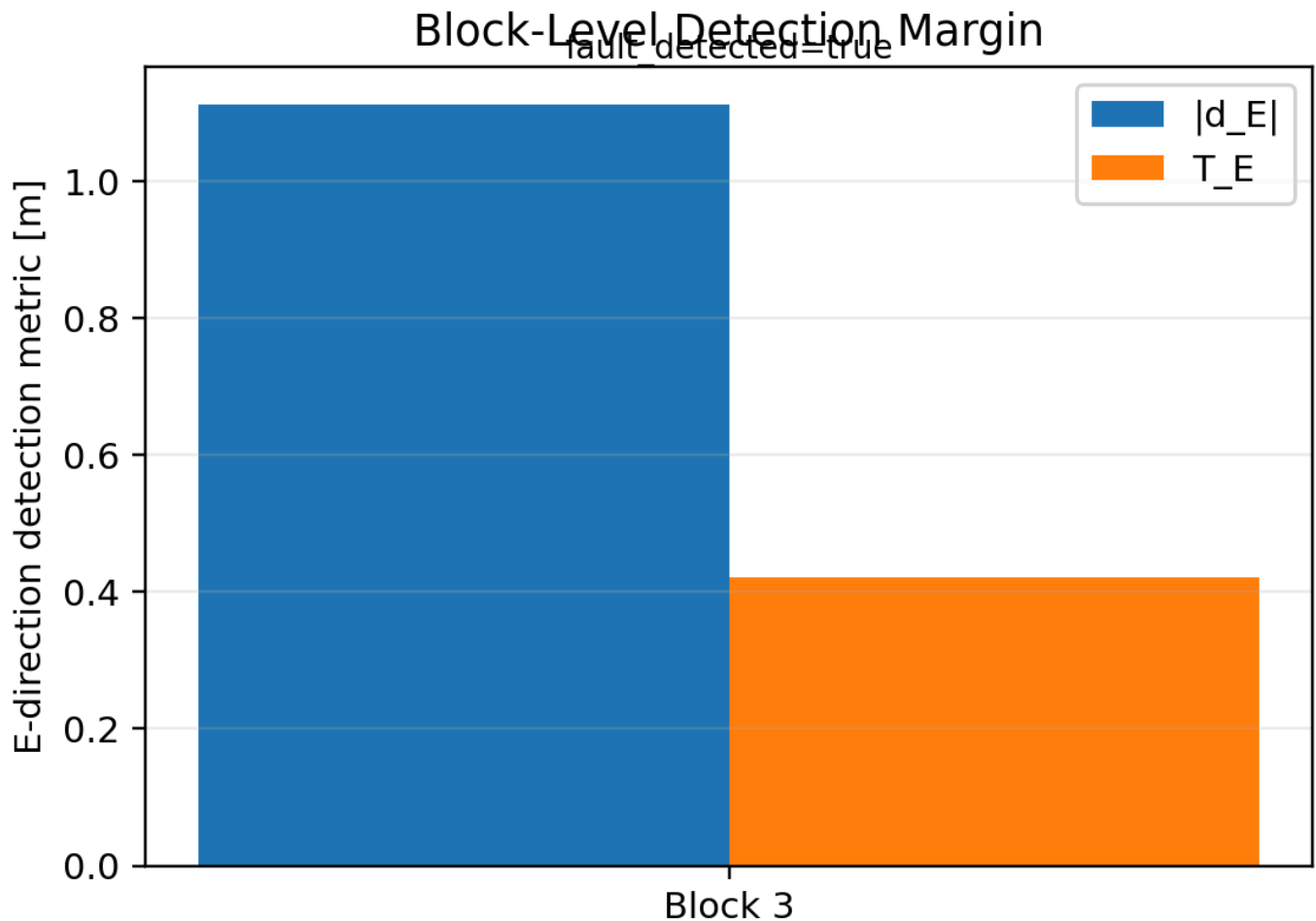
The heatmaps expose the actual 3x3 FIM matrix structure. Feature-rich information is balanced, corridor information is highly anisotropic, and sparse information is weak and ill-conditioned.

Block fault HPL/VPL:



The injected bad block raises HPL from the clean baseline while VPL stays unchanged in this E-direction fault injection.

Block-level detection margin:



For block 3,  $|d_E|$  exceeds  $T_E$ , so `fault_detected=true`. This makes the block-level diagnostic observable rather than treating the LiDAR source as a black box.

#### 15.6 Pass / Fail Criteria

| Check                                                     | Result                                                   |
|-----------------------------------------------------------|----------------------------------------------------------|
| CSV fixtures load and contain points                      | PASS                                                     |
| feature-rich FIM is valid and finite                      | PASS                                                     |
| corridor FIM is valid and finite                          | PASS                                                     |
| sparse FIM/PL is finite                                   | PASS                                                     |
| corridor condition number > feature-rich condition number | PASS: 603.304732 > 1.059827                              |
| feature-rich HPL < corridor HPL                           | PASS: 0.368296 < 7.700064                                |
| corridor HPL < sparse HPL when sparse is valid            | PASS: 7.700064 < 230.306337                              |
| corridor weak-axis PL increases                           | PASS: corridor $PL_E=7.700064$ , feature $PL_E=0.368296$ |

#### 15.7 Interpretation

Feature-rich geometry produced a nearly isotropic FIM with condition number about 1.06, so all directional PL values remained small.

Corridor geometry produced a valid but highly anisotropic FIM. The minimum eigenvalue dropped to 0.826432, condition number increased to about 603, and the horizontal E-direction PL dominated HPL:

```
corridor PL_E = 7.700064
corridor HPL = 7.700064
```

Sparse single-wall geometry produced an even weaker FIM. It remained finite and valid in the current adapter, but the condition number rose to about 45123, and PL became very large:

```
sparse HPL = 230.306337
```

```
sparse VPL = 284.228246
```

This matches the expected trend:

```
feature-rich PL < corridor PL < sparse PL
```

English conclusion:

The LiDAR Integrity Evaluator passed module-level validation under synthetic FIM and synthetic point-cloud scenarios. Feature-rich geometry produced a nearly isotropic FIM and low LiDAR PL. Corridor geometry produced a highly anisotropic FIM, with weak E-direction observability and HPL dominated by PL\_E. Sparse geometry produced an ill-conditioned FIM and very large HPL/VPL. The observed trend feature-rich PL < corridor PL < sparse PL confirms that the LiDAR integrity source responds correctly to geometric observability degradation.

□□□□□

□□□□□□□□ LiDAR source □□□□□□□□ feature-rich □□□ FIM □□□□□ 1.06□HPL/VPL □□□ 0.4 m□corridor □□□ FIM □□□□□□□ 603□E □□ PL □□□□□ HPL□sparse □□□□□□□□□□ 45123□HPL/VPL □□□□□□□□ LiDAR integrity evaluator □□□□□□□□□□□□□□□□□□

## 15.8 Build And Test Results

Commands executed from `/home/dev/ws_iap:`

```
python3 src/iap/scripts/araim_validation/generate_synthetic_lidar_clouds.py
colcon build --symlink-install --packages-select iap \
 --cmake-args -DCMAKE_BUILD_TYPE=RelWithDebInfo -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
./build/iap/test_araim --gtest_filter='LidarIntegrityValidationTest.*' --gtest_brief=1
python3 src/iap/scripts/araim_validation/plot_lidar_integrity_validation.py
colcon test --packages-select iap \
 --ctest-args -R "test_araim" --output-on-failure
colcon test-result --all
```

Results:

- Fixture generation: PASS, generated 3 CSV files, 3 individual PNG visualizations, and `lidar_cloud_inputs_comparison.png`.
- Metrics export: PASS, generated `lidar_pointcloud_fim_pl_metrics.csv` and `lidar_block_fault_metrics.csv`.
- Quantitative plotting: PASS, generated 8 measured-data figures.
- `colcon build --packages-select iap`: PASS.
- Focused `LidarIntegrityValidationTest.*`: PASS, 6 tests.
- `colcon test --packages-select iap -R "test_araim"`: PASS.
- `iap/test_araim.gtest.xml`: PASS, 86 tests, 0 failures.
- `colcon test-result --all`: nonzero because existing non-`iap` packages still report old failures; all listed `iap` tests are passing.

## 15.9 Limitations

- The C++ test uses generated CSV fixtures; it does not synthesize point clouds internally.
- `LidarObservabilityFim` is an advisory/planner-side FIM API. The PL step uses a test-local FIM-to-`LidarAraim` bridge because there is no public raw point-cloud-to-PL API.
- The sparse cloud stayed valid in this run, so the pass criterion was the finite large-PL trend rather than invalid/degraded output.
- This experiment validates deterministic synthetic geometry trends. It does not validate real sensor noise, dynamic scenes, registration convergence, or certified LiDAR ARAIM behavior.

## 16. End-To-End ROS Smoke Test: `/iap/integrity` Single Message

□□□□□2026-06-04

### 16.1 Command

After launching the end-to-end ARAIM simulation chain, inspect one runtime integrity report:

```
ros2 topic echo /iap/integrity --once
```

### 16.2 Observed Output Summary

The sampled message had:

```

stamp: 1657065669.295126199
integrity_state: 2
hpl: 23.422639821893917
vpl: 50.78597692330939
hal: 10.0
val: 20.0
im_h: -13.422639821893917
im_v: -30.78597692330939
im_min: -30.78597692330939
n_sv_used: 38
n_constellations: 4
pdop: 4.66023465479939
n_hypotheses: 45
n_detected: 0
excluded_prns: [21, 30, 140]
n_trunks_observed: 0
tdop: 1000000000.0
gnss_valid: true
lidar_valid: true
fallback_valid: true
gnss_hpl: 23.422639821893917
gnss_vpl: 50.78597692330939
lidar_hpl: 3.885729747402171
lidar_vpl: 3.883898076882984
fallback_hpl: 0.017517152523660977
fallback_vpl: 0.017517152523660977
fusion_mode: max_pl
final_hpl_source: GNSS
final_vpl_source: GNSS
final_pl_source: GNSS
failure_reason: ""

```

### 16.3 Result Assessment

The message is internally consistent for the current implementation:

- Topic existence is confirmed because `ros2 topic echo /iap/integrity --once` returned a full `iap/msg/IntegrityReport` sample.
- Field completeness for Step 1 is confirmed in this sample: primary PL/AL/IM fields, H/V margins, GNSS source fields, LiDAR source fields, fallback source fields, final source strings, and numerical failure flags are all present.
- GNSS, LiDAR, and fallback sources are all valid.
- `fusion_mode=max_pl` chooses GNSS as final source because GNSS HPL/VPL are larger than LiDAR and fallback PL.
- The final fused result is unsafe because `HPL > HAL` and `VPL > VAL`, giving negative integrity margins.
- All reported PL and AL fields are finite.
- `failure_reason` is empty and numerical rejection flags are false.

H/V margin semantics are correct for the sampled values:

```

im_h = HAL - HPL = 10.0 - 23.422639821893917 = -13.422639821893917
im_v = VAL - VPL = 20.0 - 50.78597692330939 = -30.78597692330939
im_min = im = min(im_h, im_v) = -30.78597692330939

```

Because both horizontal and vertical margins are negative, `integrity_state: 2 (UNSAFE)` is the expected state.

### 16.4 `excluded_prns` And `n_detected`

Observed:

```

n_detected: 0
gnss_n_det: 0
excluded_prns: [21, 30, 140]

```

This looks contradictory if `excluded_prns` is interpreted as "satellites excluded by ARAIM FDE only". Code inspection shows the current implementation has broader semantics:

- `n_detected / gnss_n_det` come from `GnssAraimResult::n_detected`, incremented only when ARAIM FDE marks a faulted subset.
- `excluded_prns` is populated from `IntegrityReport::excluded_sats`.
- `IntegrityMonitor::run_gnss_gating()` clears `excluded_sats`, then adds satellites that are already marked `sat.excluded` and satellites downweighted by NIS gating.
- `GnssExtensionModule::on_smoother_update_finish()` can also mark a satellite `excluded=true` when no finite pseudorange residual is available.
- The ROS mapping clears `msg.excluded_prns` before filling it, so this is not a stale field.

Therefore, in this output `n_detected=0` means no ARAIM FDE fault was detected, while `excluded_prns` currently means GNSS rejected/downweighted PRNs from prefiltering, residual availability, or NIS gating. This is a naming/semantic ambiguity, not a runtime failure.

Recommended follow-up: split or rename the fields so the message distinguishes:

```
fault_excluded_prns
prefiltered_or_gated_prns
```

Until then, reports and papers should not describe `excluded_prns` as ARAIM FDE-only exclusions.

## 16.5 `tdop = 1e9`

Observed:

```
n_trunks_observed: 0
tdop: 1000000000.0
```

Code inspection confirms `tdop` is Tree/Trunk DOP, not GNSS time DOP. `TrunkDetectionResult::tdop` defaults to `1e9`, and `TrunkDetector::Params::tdop_inf` documents this sentinel as the value used when fewer than two trunks are detected.

Therefore this sample is valid: `tdop=1e9` means no useful trunk geometry was observed in the integrity report. It does not indicate GNSS TDOP failure.

## 16.6 Fallback PL

Observed:

```
fallback_hpl: 0.017517152523660977
fallback_vpl: 0.017517152523660977
fallback_valid: true
```

Code inspection confirms the fallback source is covariance-derived:

```
fallback PL = compute_PL_proxy(frame)
```

It is based on the frame/FGO position covariance (`sigma_p`), not a conservative "all integrity sources failed" bound. In this `max_pl` run this is acceptable because GNSS dominates final HPL/VPL. However, fallback-only experiments must treat this carefully: a very small fallback PL only means the smoother covariance proxy is small; it is not automatically a certified conservative failure bound.

Recommended follow-up: split this concept in future messages or documentation:

```
covariance_fallback_hpl
covariance_fallback_vpl
conservative_failure_hpl
conservative_failure_vpl
```

## 16.7 Conclusion

The sampled `/iap/integrity` message is acceptable as an end-to-end ROS smoke-test result for the current implementation. The main caveat is field naming and documentation:

- `excluded_prns` currently contains GNSS gating/prefilter exclusions as well as any ARAIM fault exclusions.
- `tdop=1e9` is a trunk-geometry sentinel.
- fallback PL is covariance-derived and should not be interpreted as a conservative all-invalid safety bound.

## 17. Pseudorange Noise Construction In The Runtime Experiment

This note records the code-level semantics of pseudorange noise for the end-to-end ROS ARAIM experiments.

### 17.1 Launch Preset Parameters

`test_araim.launch.py` now drives both the GNSS simulator and the IAP runtime GNSS config from the same launch arguments:

```
gnss_pr_noise_base
gnss_dop_noise_base
```

For the nominal open-sky experiment:

```
experiment:=gnss_open_sky
gnss_pr_noise_base = 0.3 m
gnss_dop_noise_base = 0.03 m/s
```

For degraded, fault, outage, and full-transition experiments:

```
gnss_pr_noise_base = 5.0 m
gnss_dop_noise_base = 0.5 m/s
```

This split is intentional. Experiment 1 is a low-noise open-sky nominal case that should recover to **SAFE**; later experiments use larger measurement noise and explicit degradation/fault mechanisms to stress the integrity monitor.

## 17.2 GNSS Simulator Measurement Noise

In `gnss_sim_node`, the launch parameter is received as:

```
pseudorange_noise_std_m
```

The simulator then computes the published per-satellite measurement standard deviation as:

```
pr_std = max(pseudorange_noise_std_m * sat.psr_std_scale, 0.05)
```

The raw pseudorange measurement is generated as:

```
psr_raw =
 geometric_range
+ sagnac
+ receiver_clock_bias
+ ionosphere
+ troposphere
+ satellite_group_delay
- satellite_clock_bias
+ psr_extra_bias
+ N(0, pr_std)
```

The simulator writes:

```
obs->psr_std = {pr_std}
```

Environment effects in the simulator are not modeled as a simple additive variance term. They change visibility, bias, and sometimes the sigma scale:

- map/skymask blocked satellite:
  - if NLOS is disabled, the satellite is not published;
  - if NLOS is enabled, it is kept as NLOS.
- NLOS:
  - adds a positive pseudorange bias sampled from  $N(\text{nlos\_bias\_mean\_m}, \text{nlos\_bias\_std\_m})$ ;
  - multiplies `psr_std_scale` by 4.0;
  - degrades C/N0.
- multipath:
  - adds a sinusoidal pseudorange bias: `multipath_amp_m * sin(...)`;
  - does not currently increase `psr_std_scale`.
- fault injection:
  - either drops the satellite or adds configured pseudorange bias/ramp;
  - does not currently increase `psr_std_scale` by itself.

Therefore, in the simulator, the effective published sigma is:

```
open sky: pr_std = max(pr_noise_base, 0.05)
NLOS: pr_std = max(4 * pr_noise_base, 0.05)
```



with additional biases added separately for NLOS, multipath, or injected fault.

### 17.3 IAP Runtime Factor Noise

`GnssExtensionModule::on_range_meas_()` reads `obs->psr_std` into `sat.pr_sigma` when it is present. However, the actual GTSAM pseudorange factor noise is created later in `GnssHandler::get_factors()` using:

```
sigma_pr = pr_sigma(sat.elevation, sat.kappa)
```

After the recent open-sky fix, `GnssHandler::pr_sigma()` has this behavior:

```
s = sin(max(elevation, min_elevation))
base_sigma = pr_noise_base / s^elev_noise_exp

if kappa <= 1e-6:
 sigma_pr = max(base_sigma, 0.05)
else:
 sigma_pr = max(base_sigma, sigma_eff_canopy(kappa, elevation))
```

The canopy model is:

```
sigma_eff_canopy =
 sqrt(
 sigma_0^2
 + sigma_mp^2 / sin(elevation)^2
 + sigma_c^2 * exp(alpha * kappa / sin(elevation))
)
```

Default canopy parameters are:

```
sigma_0 = 1.0 m
sigma_mp = 0.5 m
sigma_c = 5.0 m
alpha = 2.0
```

So the IAP factor sigma is not currently "base plus obstruction". It is:

```
open-sky: elevation-weighted base sigma
canopy: max(elevation-weighted base sigma, canopy sigma)
```

This avoids the old problem where `kappa=0` still inherited the `sigma_c=5m` canopy floor and made nominal open-sky HPL too large.

### 17.4 Important Runtime Caveat

`SatObs::kappa` exists and `VisibilityPredictor` can compute line-of-sight occupancy ratio from a local occupancy grid. However, in the current end-to-end `/iap/integrity` runtime chain, `GnssExtensionModule::on_range_meas_()` does not populate `sat.kappa` from the simulator or the occupancy grid.

That means the current runtime factor noise normally follows the open-sky branch:

```
sigma_pr = pr_noise_base / sin(elevation)^elev_noise_exp
```

Environment degradation in the formal ROS experiments is therefore mainly represented by the simulator through satellite visibility, NLOS/multipath/fault biases, and the published `psr_std`. The factor noise path is still controlled by the configured `pr_noise_base` and elevation weighting unless `kappa` is explicitly populated in a future change.

ARAIM then uses the post-optimization factor sigma stored back into the epoch when building weights:

```
W_i = 1 / sigma_i^2
```

Therefore, the PL/HPL/VPL values are sensitive to:

- `gnss_pr_noise_base`;
- satellite elevation distribution;
- `elev_noise_exp`;

- number of usable satellites and constellations;
- NLOS/fault/multipath residuals through FDE and gating;
- future **kappa** population if canopy-aware factor noise is enabled end to end.

17.5 Practical Interpretation

For experiment 1, **pr\_noise\_base=0.3m** is a deliberately low-noise open-sky setting. It is suitable for proving that the runtime GNSS ARAIM chain can produce:

```
GNSS valid
final source = GNSS
HPL < HAL
VPL < VAL
integrity_state = SAFE
```

For degraded/fault/outage experiments, keep **pr\_noise\_base=5.0m** or larger and enable the corresponding scenario mechanisms. Those experiments are intended to show risk growth, unsafe margins, source switching, conservative behavior, or fault detection/exclusion rather than nominal SAFE behavior.

18. GNSS Open-Sky: Constellation-Wide Hypothesis Effect

This section compares two **gnss\_open\_sky** end-to-end bags:

- With constellation-wide hypotheses: **test\_araim\_gnss\_open\_sky\_20260604T074459Z\_with\_constellation\_hypo**
- Without constellation-wide hypotheses: **test\_araim\_gnss\_open\_sky\_20260604T135219Z\_without\_constellation\_hypo**

The purpose is to explain why the earlier open-sky experiment produced large PL values even though the nominal GNSS geometry itself was not poor.

18.1 Direct Bag-Level Comparison

| Case                              | n_sv_used<br>mean | n_hypotheses<br>mean | HPL<br>mean | VPL<br>mean | HAL    | VAL    | im_min<br>mean | Final HPL<br>source |
|-----------------------------------|-------------------|----------------------|-------------|-------------|--------|--------|----------------|---------------------|
| constellation hypotheses enabled  | 41.0              | 45.0                 | 23.41 m     | 48.00 m     | 10.0 m | 20.0 m | -28.00 m       | GNSS                |
| constellation hypotheses disabled | 40.99             | 41.0                 | 5.08 m      | 14.55 m     | 10.0 m | 20.0 m | +4.86 m        | GNSS                |

The hypothesis count confirms that the switch is effective:

```
enabled: n_hypotheses = 45 = 41 satellite hypotheses + 4 constellation hypotheses
disabled: n_hypotheses = 41 = satellite hypotheses only
```

Relevant plots:

Fig 01: PL / AL / IM Timeline

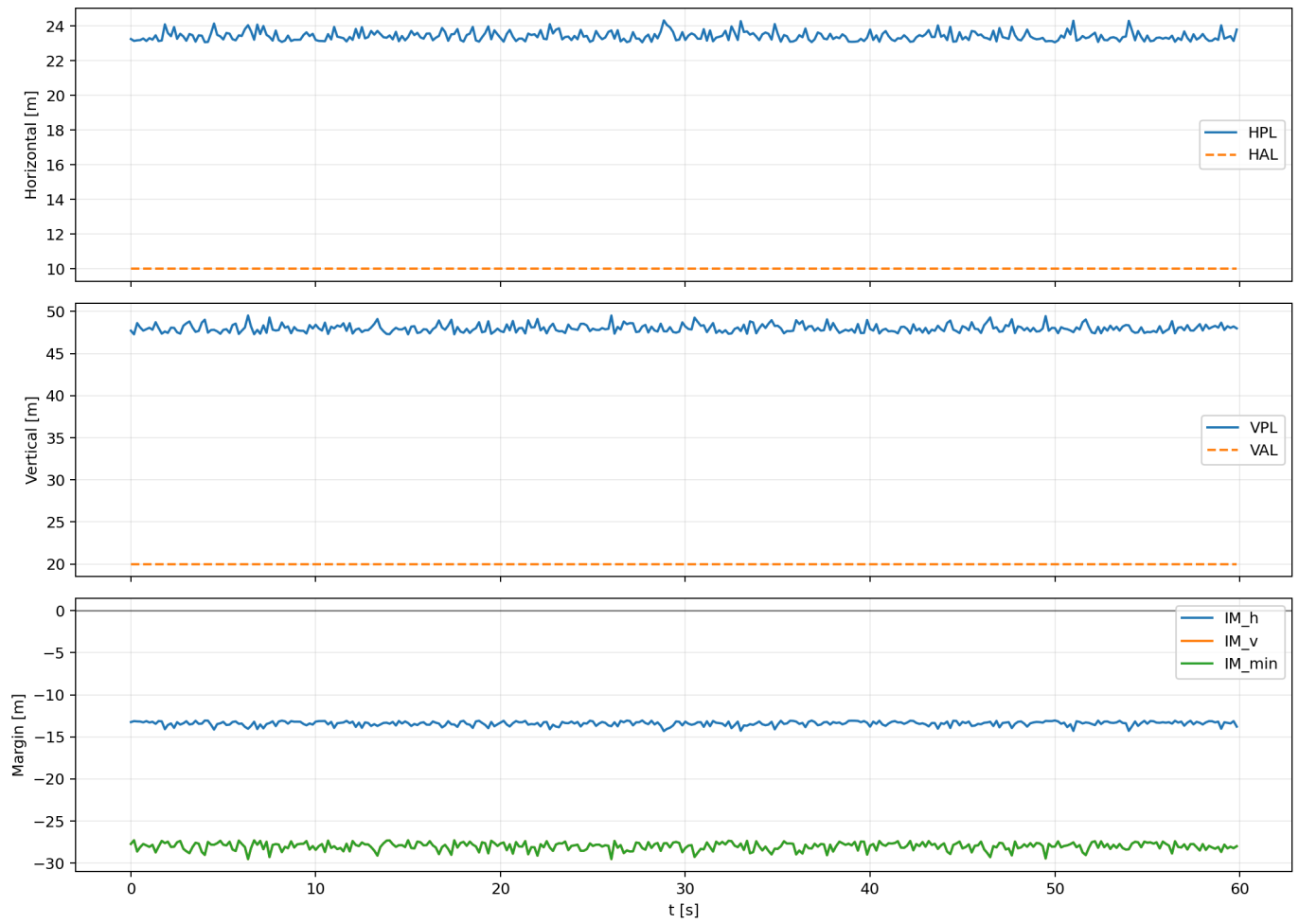
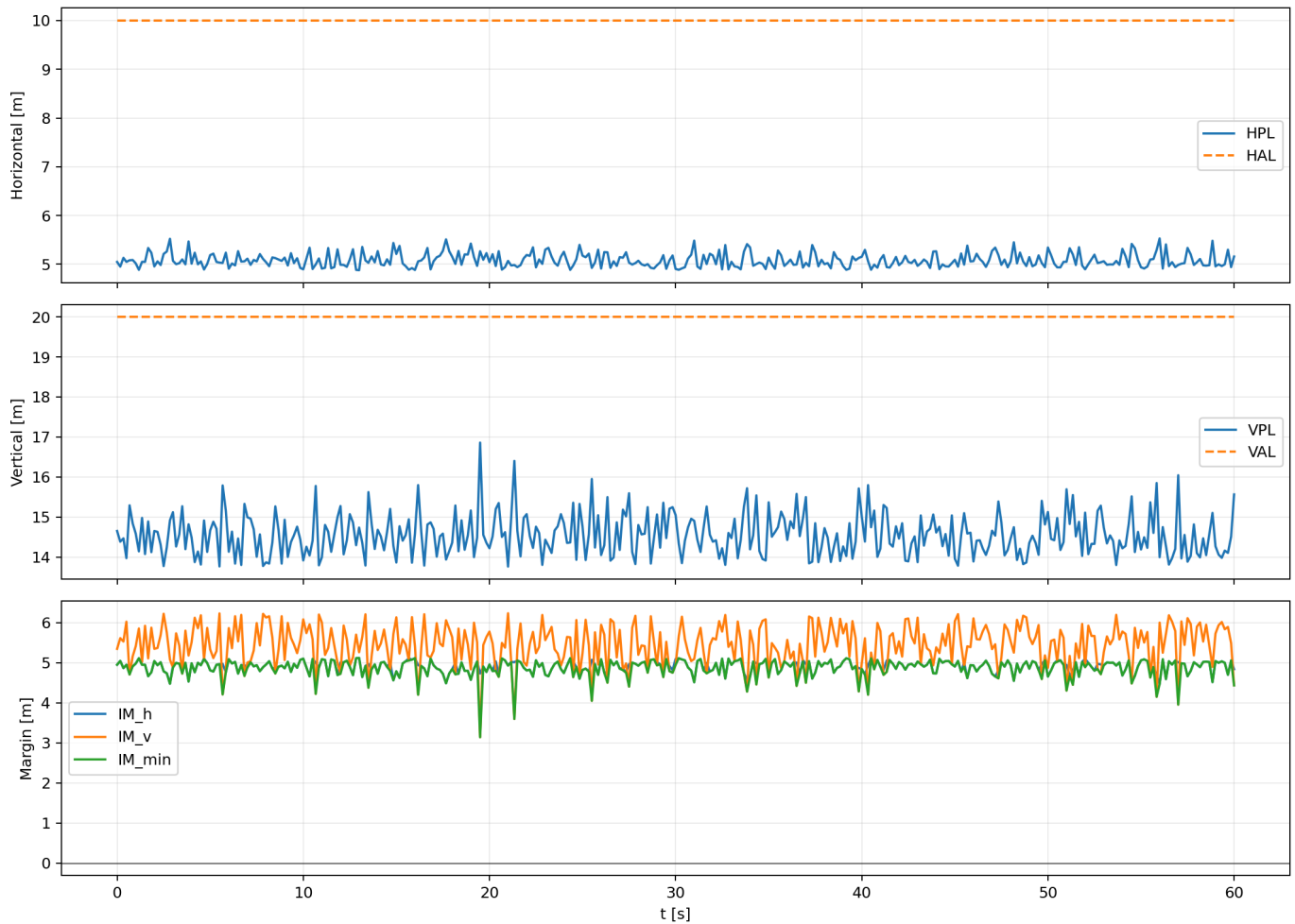


Fig 01: PL / AL / IM Timeline



The important conclusion is that disabling constellation-wide hypotheses makes the open-sky PL/AL margins positive:

```
HPL = 5.08 m < HAL = 10.0 m
VPL = 14.55 m < VAL = 20.0 m
```

In this bag, `integrity_state_counts` still reports `UNSAFE`. That should be treated as a state-machine/recovery-field issue separate from the instantaneous PL/AL safety margin. The H/V margin semantics are positive after disabling the constellation hypotheses.

## 18.2 Fault-Free Geometry Is Not The Root Cause

From the no-constellation debug CSV:

```
sigma_H,0 = 1.151 m
sigma_E,0 = 0.856 m
sigma_N,0 = 0.770 m
sigma_V,0 = 2.257 m
K_ff = 5.451
```

Using a horizontal-norm approximation:

```
HPL_ff ~ K_ff * sigma_H,0 = 5.451 * 1.151 ~ 6.28 m
VPL_ff ~ K_ff * sigma_V,0 = 5.451 * 2.257 ~ 12.30 m
```

Both are below the alert limits:

```
HPL_ff < HAL = 10 m
VPL_ff < VAL = 20 m
```

The runtime GNSS ARAIM implementation computes horizontal PL per axis and then takes `max(PL_E, PL_N)`. Therefore its fault-free horizontal PL is even lower:

```
max(sigma_E,0, sigma_N,0) * K_ff ~= 0.856 * 5.451 ~= 4.67 m
```

So open-sky GNSS geometry is not intrinsically unsafe. The unsafe result in the constellation-enabled run comes from the fault-hypothesis threat model.

### 18.3 BDS Constellation-Fault Subset Was The Dominant Cause

The PL decomposition debug run identified the dominant fault mode as the BDS constellation-wide hypothesis (`const_id=2`, `dominant_const_name=BDS`). Under that subset, removing the BDS constellation worsens the covariance-derived geometry:

```
HDOP_full = 1.15 -> HDOP_subset = 1.64
VDOP_full = 2.25 -> VDOP_subset = 3.37
```

This does not indicate a detected real fault. It indicates that the constellation-wide ARAIM hypothesis asks: "if the whole BDS constellation were faulted and removed, how conservative must the certified PL be?"

For horizontal PL, the average dominant-row decomposition was:

| Term                           | Value    | Share | Interpretation                                |
|--------------------------------|----------|-------|-----------------------------------------------|
| <code>abs(d_H)</code>          | 0.634 m  | 5.6%  | subset separation, not dominant               |
| <code>K_fa * sigma_ss,H</code> | 5.215 m  | 46.1% | constellation-removal detection threshold     |
| <code>K_md * sigma_H,k</code>  | 5.460 m  | 48.3% | BDS-removed subset covariance                 |
| <code>b_H</code>               | 0.000 m  | 0.0%  | no explicit GNSS bias term in current formula |
| total                          | 11.309 m | 100%  | HPL exceeds HAL=10 m                          |

For vertical PL, the effect was stronger:

| Term                           | Value    | Share | Interpretation                                |
|--------------------------------|----------|-------|-----------------------------------------------|
| <code>abs(d_V)</code>          | 1.018 m  | 3.6%  | subset separation, not dominant               |
| <code>K_fa * sigma_ss,V</code> | 12.938 m | 45.8% | BDS-removed separation sigma term             |
| <code>K_md * sigma_V,k</code>  | 14.279 m | 50.6% | BDS-removed vertical subset covariance        |
| <code>b_V</code>               | 0.000 m  | 0.0%  | no explicit GNSS bias term in current formula |
| total                          | 28.235 m | 100%  | VPL exceeds VAL=20 m                          |

Numerically:

```
sigma_ss,V = 2.498 m, K_fa = 5.180
K_fa * sigma_ss,V = 5.180 * 2.498 ~= 12.94 m

sigma_V,k = 3.367 m, K_md = 4.241
K_md * sigma_V,k = 4.241 * 3.367 ~= 14.28 m
```

These two terms alone contribute about:

```
12.94 m + 14.28 m = 27.22 m
```

Therefore the large VPL is not caused by bias, numerical failure, NaN/Inf, or a detected real GNSS fault. It is caused by the BDS constellation-wide fault hypothesis combined with conservative ARAIM multipliers and degraded vertical subset geometry.

The no-constellation run confirms the interpretation. The dominant hypotheses become single-satellite hypotheses only:

```
hyp_type_counts: GNSS_SAT = 722
dominant_const_name_counts: UNKNOWN = 722
n_removed_by_hyp = 1
n_remaining_after_hyp = 40
```

and the averaged PL decomposition drops to:

```
H axis: 0.217 + 1.541 + 3.326 + 0 = 5.085 m
```

V axis:  $0.858 + 4.753 + 8.943 + 0 = 14.554$  m

Relevant no-constellation diagnostic plots:

Fig 12: GNSS ARAIM PL Decomposition Terms

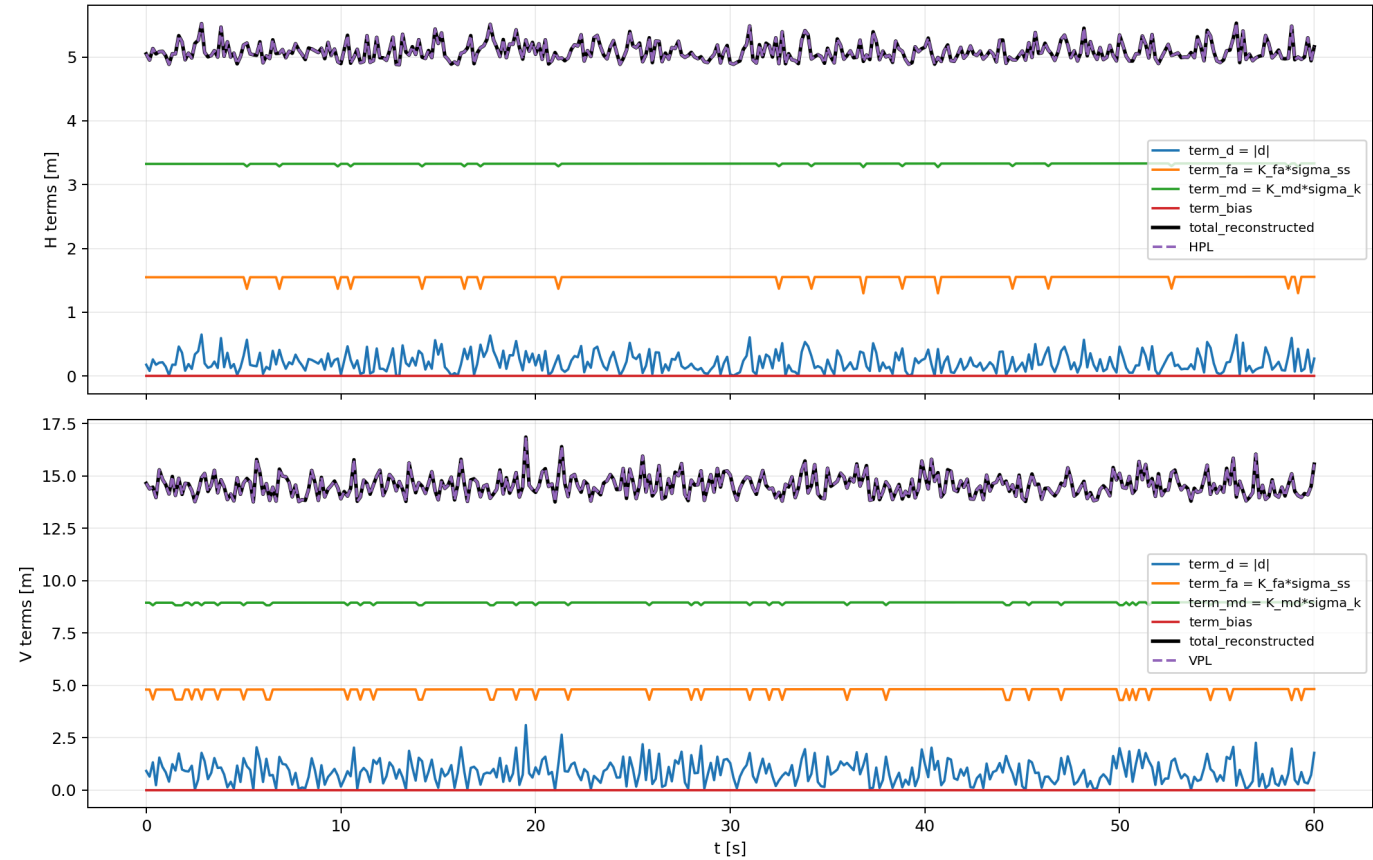


Fig 13: GNSS ARAIM Dominant Hypothesis Timeline

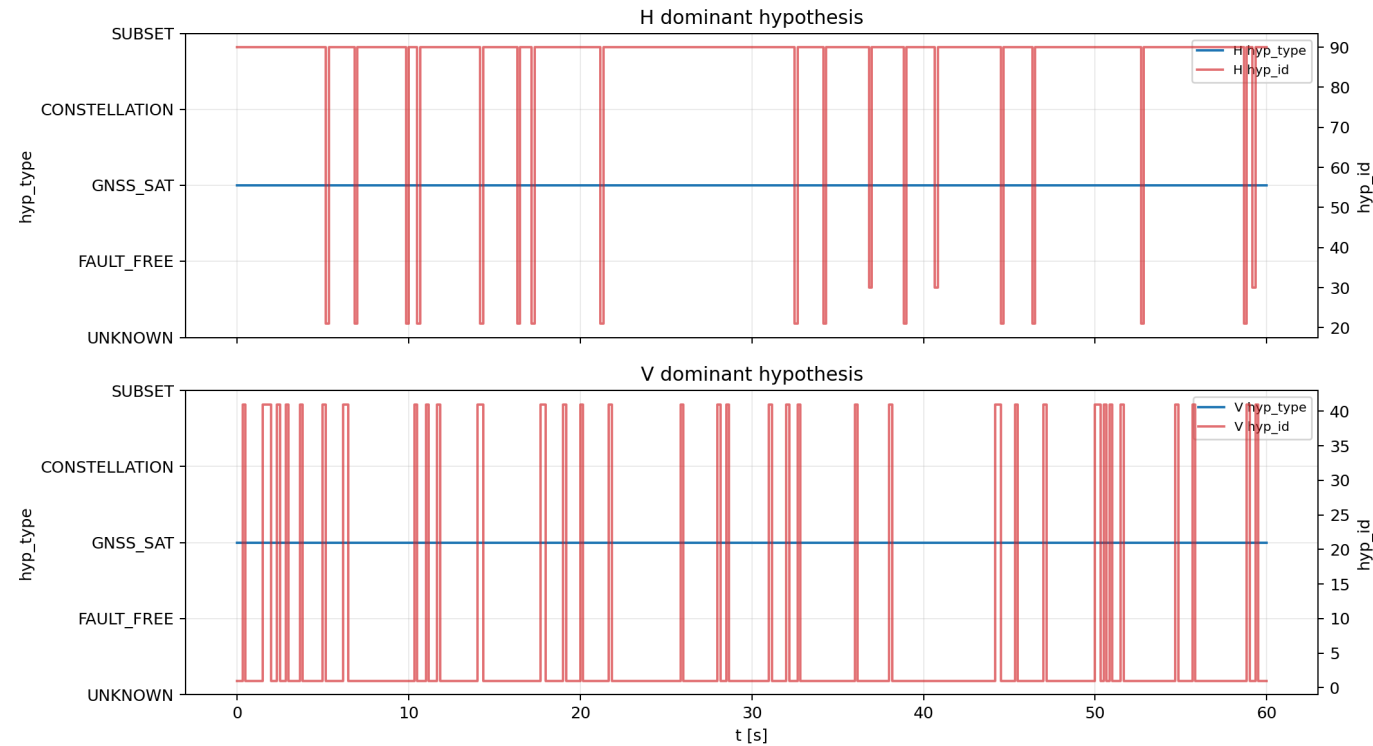
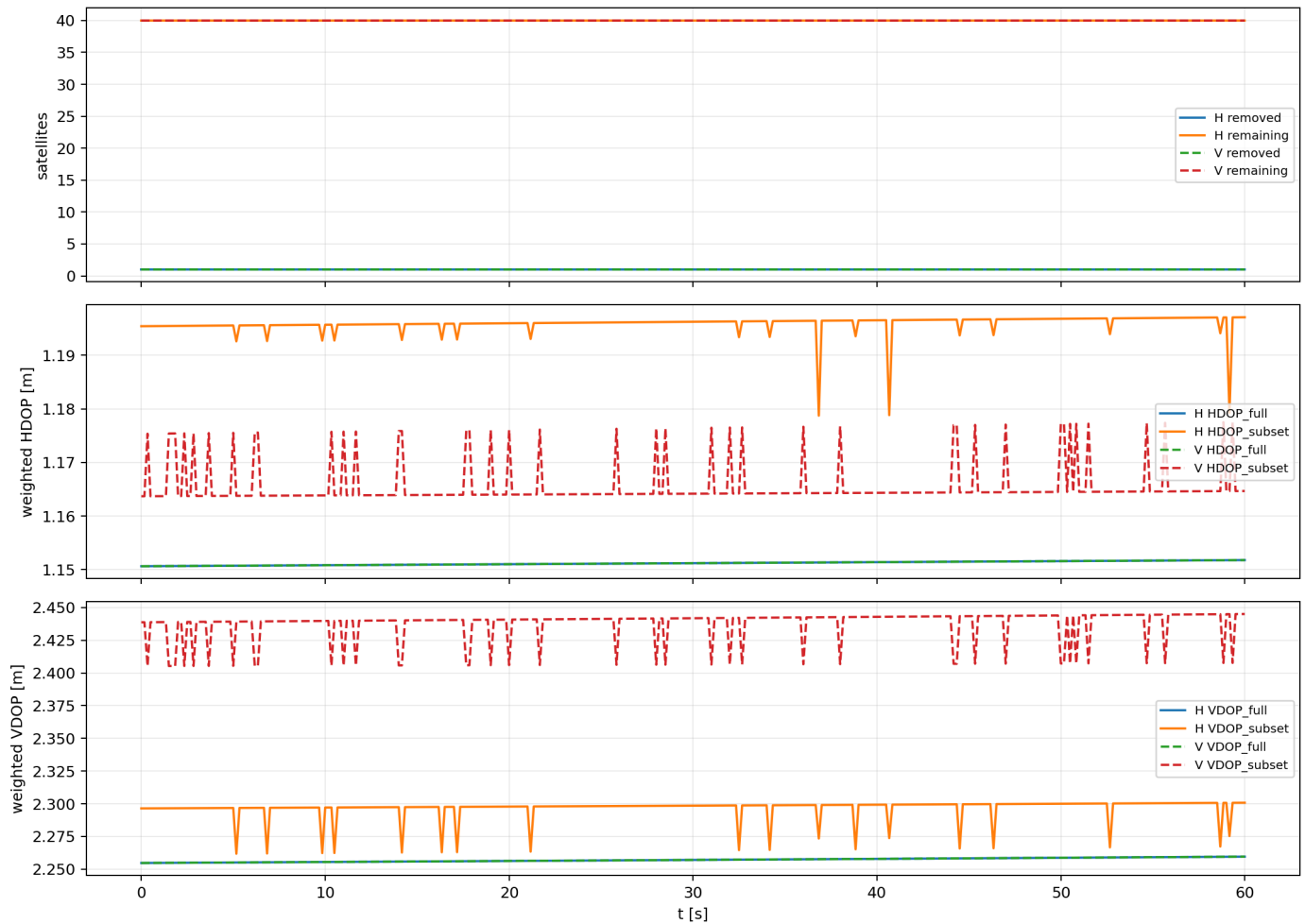


Fig 14: Constellation Removal Geometry Timeline (weighted covariance-derived)



## 18.4 Experiment Policy

For the nominal `gnss_open_sky` experiment, the default is now:

```
"gnss_araim_enable_constellation_hypotheses": false
```

This default is intentional. The open-sky experiment is meant to validate the runtime GNSS ARAIM chain under nominal conditions:

```
GNSS valid
final source = GNSS
HPL < HAL
VPL < VAL
positive H/V integrity margins
```

Constellation-wide hypotheses remain useful, but they are a separate availability/threat-model stress test. They should be re-enabled in later experiments specifically designed to evaluate conservative constellation-fault availability, not in the nominal open-sky smoke test.

## 19. Experiment 2: LiDAR-Only Feature-Rich Runtime Validation

2026-06-06

### 19.1 Purpose

This experiment validates that the LiDAR integrity runtime works in a feature-rich point-cloud environment. The run uses the `lidar_feature_rich` preset, disables GNSS integrity, enables LiDAR integrity, and requires the final runtime integrity source to be LiDAR.

Completion is evaluated according to the `lidar_feature_rich` validator preset in `test_araim.launch.py`: LiDAR must become valid, fallback must become valid, the fusion mode must remain `lidar_only`, and the final H/V PL source must be `LIDAR`. The validator does not require every frame to be `SAFE`; transient negative margins are recorded as availability/margin events rather than runtime source failures.

### 19.2 Commands

Runtime command:

```
ros2 launch iap test_araim.launch.py experiment:=lidar_feature_rich enable_araim_pl_decomp_csv:=true record_bag:=true
```

Recorded rosbag:

```
src/iap/results/araim_validation/real_time_test/test_araim_lidar_feature_rich_20260606T124316Z
```

Analysis command:

```
source install/setup.bash
python3 src/iap/test/araim_validation/analyze_araim_rosbag.py
src/iap/results/araim_validation/real_time_test/test_araim_lidar_feature_rich_20260606T124316Z --no-show
```

Analyzer result:

```
Wrote analysis to:
/home/dev/ws_iap/src/iap/results/araim_validation/real_time_test/test_araim_lidar_feature_rich_20260606T124316Z/araim_analysis
Integrity rows: 317
PL decomposition rows: 0
Figures: 13
```

The LiDAR-only run did not produce GNSS PL decomposition rows. This is expected for this experiment and does not affect the LiDAR runtime completion check.

19.3 Runtime Evidence

The rosbag analysis produced 317 `/iap/integrity` samples over about 67 s. The recorded environment topics were present, including global cloud, trunk cloud, canopy cloud, terminal wall cloud, desired trajectory, IAP odometry, and truth odometry.

Point-cloud evidence:

| Cloud         | Sampled points in analysis |
|---------------|----------------------------|
| global        | 131705                     |
| trunks        | 36928                      |
| canopy        | 60988                      |
| terminal wall | 33789                      |

Integrity-source summary:

| Check                                     | Observed result | Status                        |
|-------------------------------------------|-----------------|-------------------------------|
| <code>/iap/integrity</code> message count | 317             | PASS                          |
| <code>fusion_mode=lidar_only</code>       | 317/317         | PASS                          |
| <code>lidar_valid=true</code>             | 317/317         | PASS                          |
| <code>fallback_valid=true</code>          | 317/317         | PASS                          |
| final H/V/PL source = <code>LIDAR</code>  | 317/317         | PASS                          |
| <code>gnss_valid=false</code>             | 317/317         | PASS, expected for LiDAR-only |
| invalid flags                             | all false       | PASS                          |
| numeric NaN / Inf                         | none            | PASS                          |

Protection-level summary:

| Field        | Min       | Mean    | Max      |
|--------------|-----------|---------|----------|
| HPL          | 2.079 m   | 3.127 m | 15.378 m |
| VPL          | 2.116 m   | 3.208 m | 31.576 m |
| IM_min       | -11.576 m | 6.826 m | 7.921 m  |
| fallback HPL | 0.019 m   | 0.029 m | 0.046 m  |
| fallback VPL | 0.019 m   | 0.029 m | 0.046 m  |



Integrity state counts:

| State  | Count |
|--------|-------|
| SAFE   | 315   |
| UNSAFE | 2     |

The two UNSAFE frames were:

| t [s]   | Cause                      | Values                                                  |
|---------|----------------------------|---------------------------------------------------------|
| 0.0000  | horizontal margin negative | HPL 15.378 m > HAL 10.000 m; VPL 7.585 m < VAL 20.000 m |
| 12.9999 | vertical margin negative   | HPL 6.494 m < HAL 10.000 m; VPL 31.576 m > VAL 20.000 m |

Both UNSAFE frames still had `lidar_valid=true`, `fallback_valid=true`, `fusion_mode=lidar_only`, and final source `LIDAR`. They therefore indicate temporary LiDAR PL/AL margin exceedance, not LiDAR integrity runtime failure.

Runtime log evidence is consistent with the rosbag:

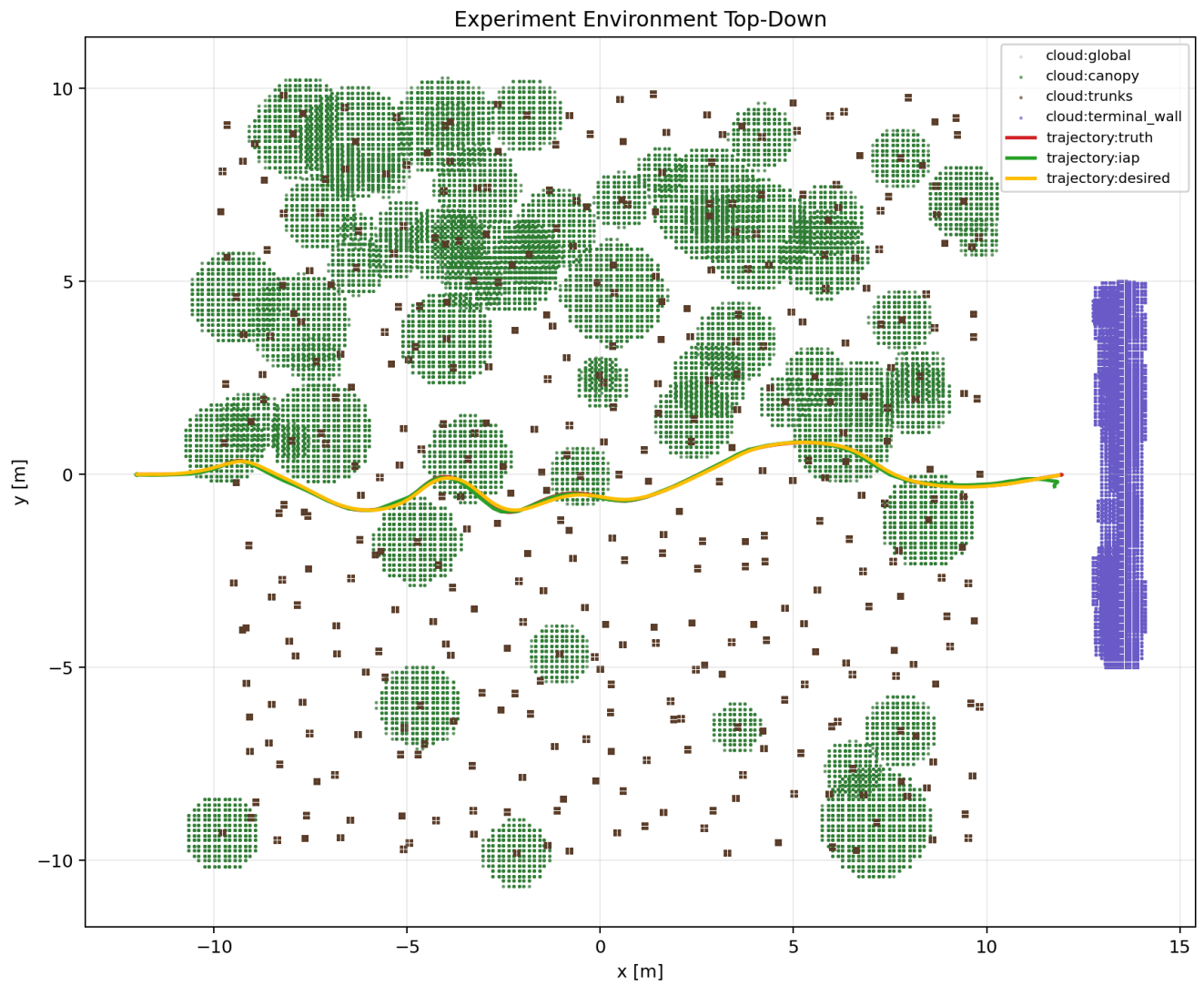
```
enable_gnss_integrity = false
enable_lidar_integrity = true
integrity_fusion_mode = lidar_only
LiDAR ARAIM Stage0 CSV: ENABLED
```

Timing data also shows the LiDAR integrity path executed for every integrity sample:

| Module              | Count | Min      | Mean     | Max      |
|---------------------|-------|----------|----------|----------|
| 2.2_lidar_araim     | 317   | 0.013 ms | 0.020 ms | 0.039 ms |
| 2.3_integrity_total | 317   | 0.029 ms | 0.043 ms | 0.096 ms |

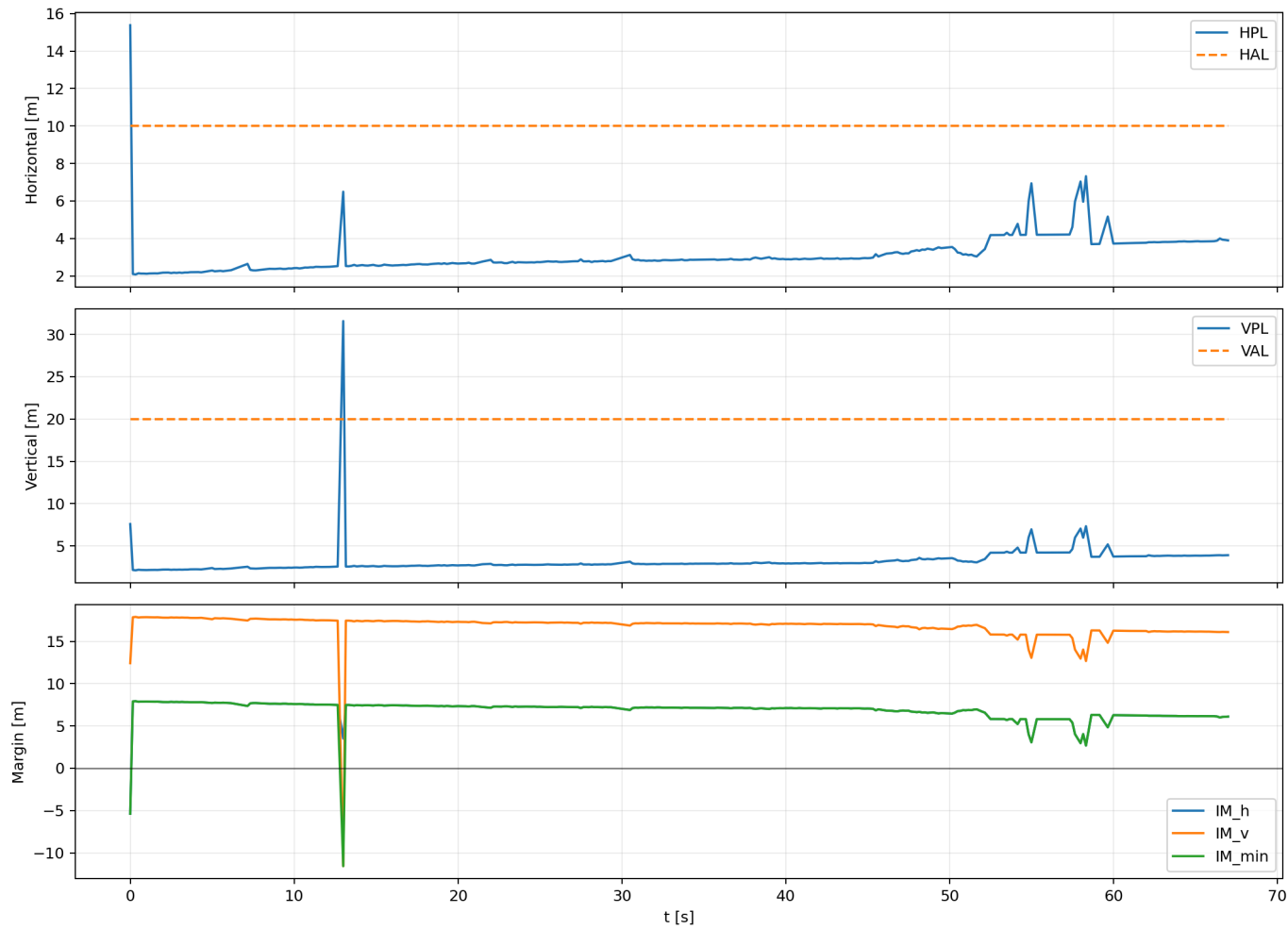
19.4 Diagnostic Plots

Feature-rich environment and trajectory:



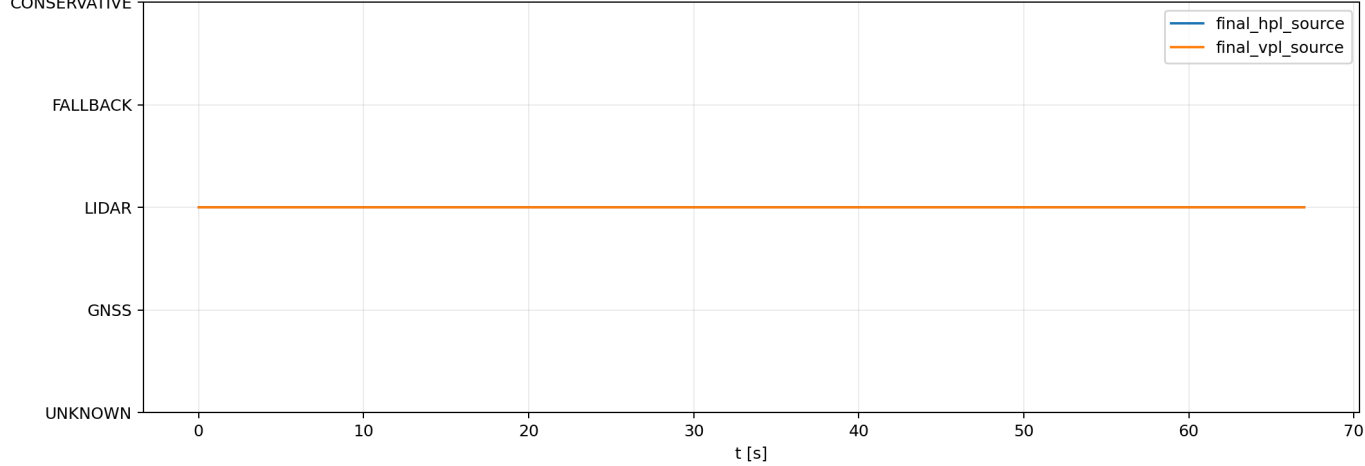
PL/AL/IM timeline:

Fig 01: PL / AL / IM Timeline



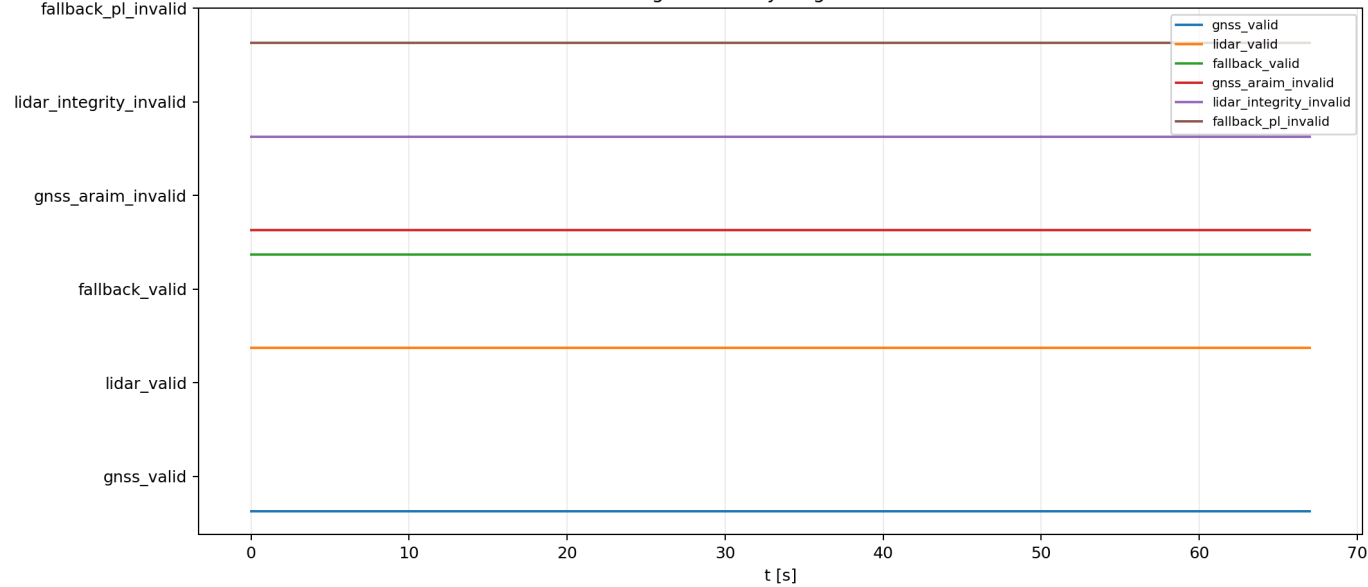
Final source timeline:

Fig 04: Final Source Timeline



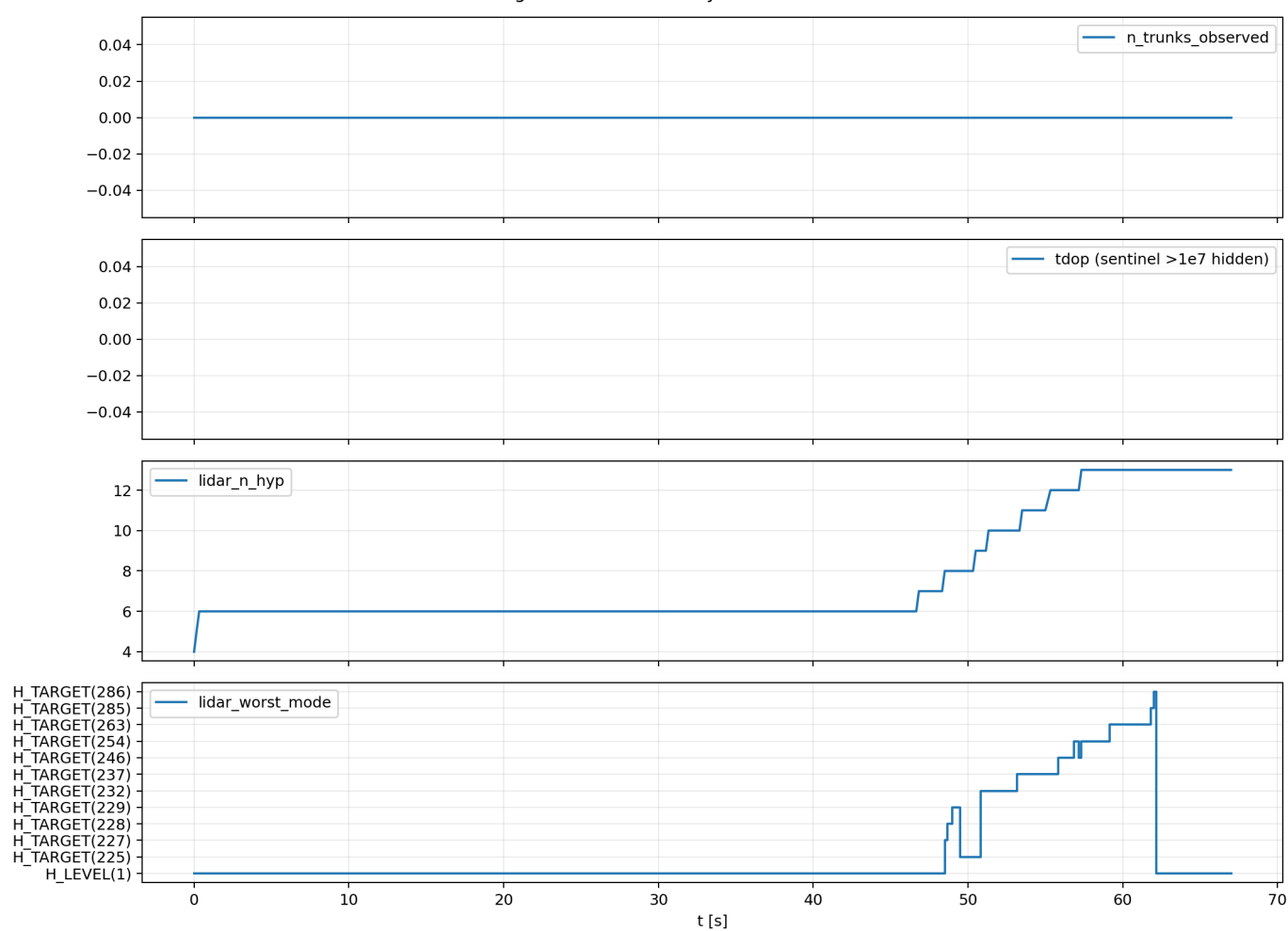
Validity flags:

Fig 05: Validity Flags Timeline

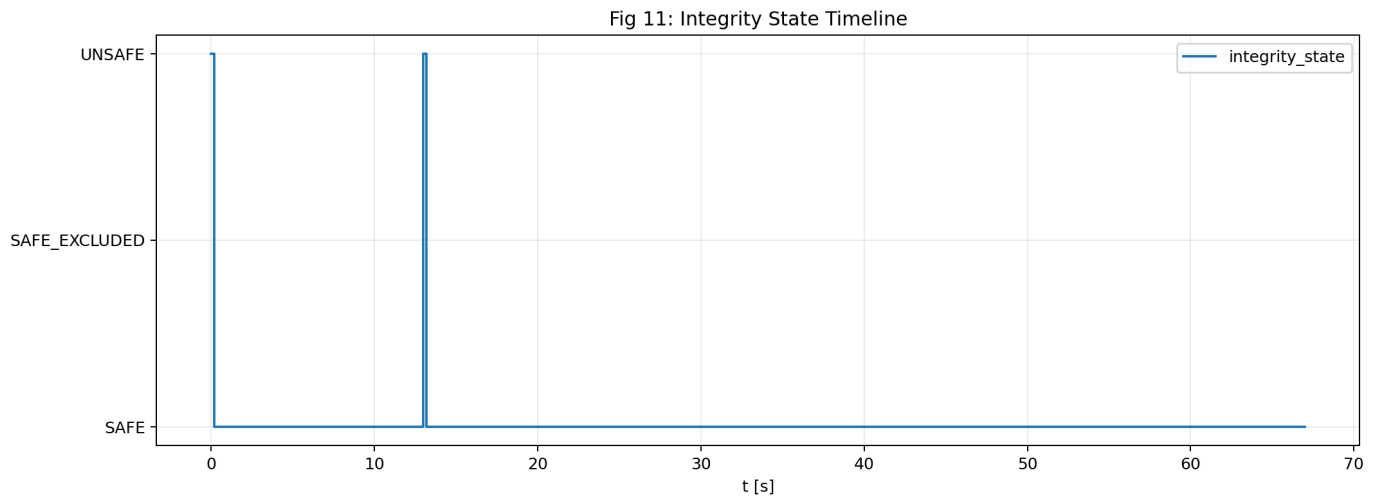


LiDAR geometry timeline:

Fig 10: LiDAR Geometry Timeline



Integrity state timeline:



### 19.5 Conclusion

Experiment 2 satisfies the LiDAR-only feature-rich runtime completion conditions. The runtime published finite `/iap/integrity` messages, selected LiDAR as the final source for every frame, kept LiDAR and fallback validity true for every frame, and reported no invalid numerical flags. The two UNSAFE frames are documented PL/AL exceedances and do not invalidate the runtime-source completion criterion.